

Sıfırdan İşlemciye

Sayısal Donanım Tasarımı: Sayı Sistemlerinden Basit Bir CPU
Mimarisine

Şafak Şahin

Sıfırdan İşlemciye — Sayısal Donanım Tasarımı

Şafak Şahin

Sürüm 0.1.0, 2026

Kişisel çalışma notu olarak hazırlanmıştır; yayıncı yoktur.

Bu belgenin LaTeX altyapısı (preamble/renk-modu/tikz-sekilleri-makrolar/sürüm-ayarları), Oğuz Er-
gin'in *Bilgisayar Mimarisi* kitabından (<https://github.com/prof-oguzergin/bilgisayar-mimarisi>) uyar-
lanmıştır; bkz. o projenin lisansı. Bu belgedeki özgün metin ve teknik içerik yazarına aittir.

İlk sürüm, Temmuz 2026

İçindekiler

Şekil Listesi	v
Tablo Listesi	vii
Bilgi Notları Listesi	ix
1 Sayı Sistemleri ve Kodlama	1
1.1 Giriş	1
1.2 Sayı Tabanları ve Dönüşümler	2
1.2.1 Pozisyonel Sayı Sistemi	2
1.2.2 İkiden Ondalığa Dönüşüm	2
1.2.3 Ondalıktan İkiliye Dönüşüm	2
1.2.4 Sekizlik ve Onaltılık Sistemler: Sembol Sorunu	4
1.2.5 Sekizlik ve Onaltılıktan Ondalığa Dönüşüm	4
1.2.6 Ondalıktan Sekizliğe ve Onaltılığa Dönüşüm	5
1.2.7 İkili, Sekizlik ve Onaltılık Arasında Gruplama	6
1.2.8 Tabanı Bilinmeyen İşlemlerde Taban Bulma	7
1.2.9 Kesirli Sayı Dönüşümleri	8
1.3 İkili Aritmetik: Toplama, Çıkarma, Çarpma ve Bölme	10
1.3.1 İkili Toplama	10
1.3.2 İkili Çıkarma	11
1.3.3 İkili Çarpma	12
1.3.4 İkili Bölme	13
1.4 Tümleyenler	14
1.4.1 Genel Tümleyen Tanımı	14
1.4.2 Ondalık Örneklerle Tümleyen Hesabı	15
1.4.3 Kısayol Yöntemi ve İspatı	15
1.4.4 İkilide Tümleyenler	16
1.4.5 Sekizlik ve Onaltılıkta $(r-1)$ 'in Tümleyeni	16
1.4.6 Kesirli (Noktalı) Sayılarda Tümleyen Alma	17
1.4.7 r 'nin Tümleyeniyle Çıkarma İşlemi	18
1.4.8 $(r-1)$ 'in Tümleyeniyle Çıkarma: Uçtan-Dolaşan Elde	19
1.5 İşaretli İkili Sayılar	20
1.5.1 İşaret Biti ve Yorumlama	20
1.5.2 İşaret-Büyüklik Gösterimi	20
1.5.3 İşaretli Tümleyen Sistemi: 1'in ve 2'nin Tümleyeni	21

1.5.4	Neden 2'nin Tümleyeni Tercih Edilir	22
1.5.5	İşaretli Toplama	23
1.5.6	İşaretli Çıkarma	24
1.6	IEEE 754 ile Kayan Noktalı Sayı Gösterimi	25
1.6.1	Neden Sabit Nokta Yetmez?	26
1.6.2	Normalizasyon: Bedava Bir Bit	26
1.6.3	IEEE 754 Formatları ve Bias	27
1.6.4	Ondalıktan IEEE 754'e Dönüşüm	27
1.6.5	IEEE 754'ten Ondalığa Dönüşüm	28
1.6.6	Özel Değerler ve Kapanış	28
1.7	İkili Kodlama Sistemleri	30
1.7.1	BCD (Binary-Coded Decimal) Kodu	30
1.7.2	BCD Toplama	31
1.7.3	Diğer Ondalık Kodlar	33
1.7.4	BCD'de İşaretli Ondalık Aritmetik (10'un Tümleyeni)	35
1.7.5	Gray Kodu	37
1.7.6	ASCII Karakter Kodu	39
1.7.7	Hata Sezme: Parity Biti	40
2	Bool Cebri ve Mantık Kapıları	43
2.1	Giriş	43
2.2	İkili Değişkenler ve Mantık Fonksiyonları	44
2.3	Bool Cebri'nin Aksiyomları	44
2.4	Düalite İlkesi ve Temel Teoremler	45
	Terimler Sözlüğü	47

Şekil Listesi

Bölüm 1 – Sayı Sistemleri ve Kodlama

Bölüm 2 – Bool Cebri ve Mantık Kapıları

Tablo Listesi

Bölüm 1 – Sayı Sistemleri ve Kodlama

1.1	Ondalık–Onaltılık–İkili Karşılık Tablosu	4
1.2	4-Bit İşaretli Sayılar: Üç Gösterimin Karşılaştırması	22
1.3	IEEE 754 Tek ve Çift Duyarlıklı Formatlar	27

Bölüm 2 – Bool Cebri ve Mantık Kapıları

2.1	Huntington Postülatları	44
2.2	Temel Teoremler	46

Bilgi Notları Listesi

Bölüm 1 – Sayı Sistemleri ve Kodlama

1.1	Neden Bilgisayarlar İkili Kullanır?	2
1.2	Neden Kalan, Bölüm Değil?	3
1.3	Neden "10" Değil de Yeni Bir Sembol?	4
1.4	Tuzak: Yön Simetrik Değildir	8
1.5	Her Kesir Sonlanmaz	9
1.6	Neden Çıkarmayı Toplamaya Çeviriyoruz?	14
1.7	Pratik Kısayol: r 'nin Tümleyenini Doğrudan Almak	15
1.8	Kısayol Neden İşliyor? – Borç Yayılımı İspatı	15
1.9	Önce Basamak Sayılarını Eşitleyin	18
1.10	Aynı Bitler, Farklı Anlam	20
1.11	İpucu: Ağırlıklı Toplamla Doğrudan Değer Okuma	21
1.12	Sebeplere 1: Tek Sıfır	22
1.13	Sebeplere 2 (Asıl Önemli Olan): Aritmetik Kolaylığı	23
1.14	Taşma (Overflow)	23
1.15	Büyük Resim: Tek Bir Devre Yeter	25
1.16	Örtük Baştaki 1	26
1.17	Neden İşaretli Değil, Biaslı Üs?	27
1.18	Artık Biliyoruz: $0.1 + 0.2 \neq 0.3$	30
1.19	Öz-Tümleyen Kodlarla Bağlantı	36
1.20	İkiliden Gray Koduna Dönüşüm	38
1.21	Tarihi Bir Not: ASCII'nin Orijinal Bit Numaralandırması	39
1.22	Neden 8 Bit (Byte) ile Saklanır?	40
1.23	Parity'nin Kritik Sınırlaması: Çift Sayıda Hatayı Yakalayamaz	41

Bölüm 2 – Bool Cebri ve Mantık Kapıları

2.1	Neden Üç Eşdeğer Gösterim?	44
2.2	Anahtarlama Devresi Yorumu	44
2.3	Sıradan Cebirden Fark	45
2.4	Düalitenin Getirisi: İkinci Teorem Bedava	46

Sayı Sistemleri ve Kodlama

Öğrenme Hedefleri

Bu bölümü tamamladığınızda aşağıdakileri yapabileceksiniz:

- Pozisyonel sayı sistemlerinin genel yapısını ve herhangi bir tabandaki sayıyı ondalığa çevirmeyi uygulamak.
- Ondalık bir sayıyı ardışık bölme yöntemiyle istenen tabana çevirmek.
- İkili, sekizlik ve onaltılık sistemler arasında doğrudan bit gruplamasıyla dönüşüm yapmak.
- İkili sayılarda toplama, çıkarma, çarpma ve bölme işlemlerini genel r -tabanlı formüllerleriyle birlikte elle yapmak.
- $(r-1)$ 'in ve r 'nin tümleyenlerini (9'un/10'un, 1'in/2'nin) hesaplamak ve çıkarma işlemini toplamaya çevirmek için kullanmak.
- İşaretli tam sayıların (işaret-büyüklik, 1'in ve 2'nin tümleyeni) gösterimlerini karşılaştırmak ve 2'nin tümleyeninin neden tercih edildiğini gerekçelendirmek.
- 2'nin tümleyeni sistemiyle işaretli toplama/çıkarma yapmak ve taşma (overflow) kavramını tanımlamak.
- IEEE 754 standardıyla kayan noktalı (ondalıklı) sayıları normalize edip işaret/üs/mantissa alanlarına ayırmak, ve $0.1+0.2 \neq 0.3$ gibi yuvarlama tuhaflıklarının kaynağını açıklamak.
- BCD, Gray kodu ve ASCII gibi temel ikili kodların kullanım amaçlarını açıklamak.

1.1 Giriş

Bir bilgisayarın belleğinde tuttuğu her şey — bir tam sayı, bir metin karakteri, bir fotoğrafın bir pikseli, bir ses örneği — sonunda aynı hammaddeye indirgenir: 0 ve 1'lerden oluşan bir dizi. Bu indirgemenin nasıl yapıldığını, yani sayıların ve sembollerin ikili düzende nasıl *kodlandığını* anlamadan, bu bitleri işleyecek devreleri tasarlamaya başlayamayız. Bu yüzden yolculuğumuza, tüm sayısal donanım tasarımının en alt katmanını olan sayı sistemleriyle başlıyoruz: önce herhangi bir tabandaki bir sayıyı nasıl okuyup dönüştüreceğimizi, ardından negatif sayıları ve metni/sembolleri ikili düzende nasıl temsil edeceğimizi göreceğiz.

1.2 Sayı Tabanları ve Dönüşümler

1.2.1 Pozisyonel Sayı Sistemi

Pozisyonel Sayı Sistemi: Bir sayı sisteminde her basamağın değeri, o basamağın *konumuna* bağlıysa buna *pozisyonel (ağırlıklı) sayı sistemi* denir. r tabanında (radix) yazılmış bir sayı, aşağıdaki ağırlıklı toplamla ifade edilir:

$$(d_n d_{n-1} \dots d_1 d_0)_r = \sum_{i=0}^n d_i \cdot r^i, \quad 0 \leq d_i < r$$

Günlük kullandığımız ondalık (decimal) sistem, $r = 10$ özel durumudur; örneğin $137 = 1 \cdot 10^2 + 3 \cdot 10^1 + 7 \cdot 10^0$.

Bilgi Notu 1.1: Neden Bilgisayarlar İkili Kullanır?

İkili (binary, $r = 2$) sistemin seçimi matematiksel değil, *fiziksel* bir mühendislik kararıdır. Bir devrede onluk sistem için gereken 10 farklı gerilim seviyesini güvenilir biçimde ayırt etmek pahalı ve gürültüye karşı kırılgan olurdu. Seviyeler arası fark küçük kalır, en ufak bir parazit yanlış okumaya yol açar. Yalnızca *iki* durumu (akım var/yok, yüksek/düşük gerilim) ayırt etmek çok daha güvenilir ve ucuzdur; seviyeler arasında büyük bir güvenlik payı (gürültü marjı) bırakılabilir. İkili değişkenler üzerine kurulu Bool cebirini Bölüm 2’de işleyeceğiz.

1.2.2 İkiliden Ondalığa Dönüşüm

Bir tabandan ondalığa dönüşüm, pozisyonel sistem tanımındaki ağırlıklı toplamın doğrudan uygulanmasıyla yapılır. Zorluğu artan üç örnekle pekiştirelim.

Örnek 1.1: $(101)_2$ ’nin Ondalık Karşılığı

$$(101)_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 4 + 0 + 1 = (5)_{10}$$

Örnek 1.2: $(1011)_2$ ’nin Ondalık Karşılığı

$$(1011)_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 8 + 0 + 2 + 1 = (11)_{10}$$

Örnek 1.3: $(110101)_2$ ’nin Ondalık Karşılığı

$$(110101)_2 = 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 32 + 16 + 0 + 4 + 0 + 1 = (53)_{10}$$

1.2.3 Ondalıktan İkiliye Dönüşüm

Ondalıktan bir tabana geçiş, ağırlıklı toplamın tersine çevrilmesiyle yapılır: tam sayı kısmı hedef tabana *ardışık olarak bölünür*; her adımın kalanı, *sondan başa* doğru okunarak sonuç

elde edilir. Şimdi bunu, zorluğu artan üç örnekle, hep ikilik tabanda pekiştirelim.

Örnek 1.4: $(6)_{10}$ 'un İkili Karşılığı

İşlem	Bölüm	Kalan
$6 \div 2$	3	0 (en düşük anlamlı bit)
$3 \div 2$	1	1
$1 \div 2$	0	1 (en yüksek anlamlı bit)

Kalanlar aşağıdan yukarı okunduğunda: $(6)_{10} = (110)_2$.

Bilgi Notu 1.2: Neden Kalan, Bölüm Değil?

$N = d_n r^n + \dots + d_1 r + d_0$ olsun. d_0 dışındaki her terim r 'nin katı olduğundan, $N \bmod r = d_0$ — yani **kalan, her zaman tam olarak en düşük anlamlı basamaktır**. Buna karşılık $\lfloor N/r \rfloor = d_n r^{n-1} + \dots + d_1$: **bölüm, aynı sayının son basamağı silinmiş hâlidir**, kendisi asla bir basamak değildir. İşlemi bölüm 0'a düşene kadar tekrarlarız (0'ı basamak olarak yazmayız); her adımda yazılan basamak o adımın *kalanıdır*, bölümü değil. Yukarıdaki örneğin ikinci satırında ($3 \div 2 = 1$ kalan 1) bölüm (1) ile kalan (1) aynı değeri taşıyor — bu, satırı yanlış okuyup bölümü basamak sanmanın fark edilmeyeceği bir durum yaratır. Aşağıdaki iki örnekte bölüm ve kalanın genelde *birbirinden bağımsız, farklı* sayılar olduğunu göreceğiz.

Örnek 1.5: $(25)_{10}$ 'un İkili Karşılığı

İşlem	Bölüm	Kalan
$25 \div 2$	12	1 (en düşük anlamlı bit)
$12 \div 2$	6	0
$6 \div 2$	3	0
$3 \div 2$	1	1
$1 \div 2$	0	1 (en yüksek anlamlı bit)

Kalanlar aşağıdan yukarı okunduğunda: $(25)_{10} = (11001)_2$.

Örnek 1.6: $(45)_{10}$ 'un İkili Karşılığı

İşlem	Bölüm	Kalan
$45 \div 2$	22	1 (en düşük anlamlı bit)
$22 \div 2$	11	0
$11 \div 2$	5	1
$5 \div 2$	2	1
$2 \div 2$	1	0
$1 \div 2$	0	1 (en yüksek anlamlı bit)

Kalanlar aşağıdan yukarı okunduğunda: $(45)_{10} = (101101)_2$. Doğrulama: $32 + 8 + 4 + 1 = 45$. ✓

1.2.4 Sekizlik ve Onaltılık Sistemler: Sembol Sorunu

Onluk sistemde tam 10 rakam sembolümüz vardır: $0, 1, \dots, 9$. Bu, $r \leq 10$ olan her taban için yeterlidir — sekizlik sistemde yalnızca 0-7 kullanılır, 8 ve 9'a hiç dokunulmaz. Ama $r = 16$ olduğunda her basamak 0'dan 15'e kadar **16 farklı değer** alabilmelidir; elimizdeki 10 rakam sembolü 6 değer için (10, 11, ..., 15) yetersiz kalır.

Bilgi Notu 1.3: Neden "10" Değil de Yeni Bir Sembol?

Akla gelen ilk çözüm, 10'u iki karakterle ("10") yazmak gibi görünebilir — ama bu, pozisyonel gösterimin temel kuralını bozar: **her basamak tek bir karakter olmalıdır**, yoksa gösterim belirsizleşir. $(110)_{16}$ dizisini gördüğünüzde bunun "1, 1, 0" mı yoksa "1, 10" mı olduğunu ayırt edemezsiniz. Çözüm, onluk rakamların bittiği yerden alfabeyle devam etmektir: $A = 10, B = 11, C = 12, D = 13, E = 14, F = 15$. Bu, matematiksel bir zorunluluk değil, *yalnızca* her basamağı tek karakterde tutmak için benimsenmiş bir gösterim kuralıdır.

Tablo 1.1: Ondalık–Onaltılık–İkili Karşılık Tablosu

Ondalık	Onaltılık	4-bit İkili
0–9	0–9	0000–1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Bu tablo (Tablo 1.1), donanım tasarımında sürekli kullanılacağı için ezberlenmeye değer.

1.2.5 Sekizlik ve Onaltılıktan Ondalığa Dönüşüm

Bu dönüşüm, Bölüm 1.2.1'de kurduğumuz ağırlıklı toplam formülünün doğrudan uygulanmasıdır. Yeni bir yöntem gerekmez. Onaltılıkta tek fark, bir harf basamağıyla karşılaştığınızda onu Tablo 1.1'daki sayısal karşılığıyla değiştirmenizdir.

Örnek 1.7: $(247)_8$ 'in Ondalık Karşılığı

$$(247)_8 = 2 \cdot 8^2 + 4 \cdot 8^1 + 7 \cdot 8^0 = 128 + 32 + 7 = (167)_{10}$$

Örnek 1.8: $(17)_8$ 'in Ondalık Karşılığı

$$(17)_8 = 1 \cdot 8^1 + 7 \cdot 8^0 = 8 + 7 = (15)_{10}$$

Örnek 1.9: $(3A)_{16}$ 'nın Ondalık Karşılığı

Harf basamağı önce sayısal değerine çevrilir: $A = 10$.

$$(3A)_{16} = 3 \cdot 16^1 + 10 \cdot 16^0 = 48 + 10 = (58)_{10}$$

Örnek 1.10: $(2FC)_{16}$ 'nın Ondalık Karşılığı

İki harf basamağı birden çevrilir: $F = 15$, $C = 12$.

$$(2FC)_{16} = 2 \cdot 16^2 + 15 \cdot 16^1 + 12 \cdot 16^0 = 512 + 240 + 12 = (764)_{10}$$

1.2.6 Ondalıktan Sekizliğe ve Onaltılığa Dönüşüm

Bu dönüşüm de yeni bir yöntem gerektirmez: Bölüm 1.2.3'de öğrendiğimiz ardışık bölme yöntemi, taban 2 yerine 8 veya 16 ile aynen uygulanır. Onaltılıkta tek fark, bir kalan 10 veya üzerine çıktığında Tablo 1.1 ile harfe çevrilmesidir.

Örnek 1.11: $(100)_{10}$ 'un Sekizlik Karşılığı

İşlem	Bölüm	Kalan
$100 \div 8$	12	4 (en düşük anlamlı basamak)
$12 \div 8$	1	4
$1 \div 8$	0	1 (en yüksek anlamlı basamak)

Kalanlar aşağıdan yukarı okunduğunda: $(100)_{10} = (144)_8$. Doğrulama: $1 \cdot 8^2 + 4 \cdot 8^1 + 4 \cdot 8^0 = 64 + 32 + 4 = 100$. ✓

Örnek 1.12: $(45)_{10}$ 'un Sekizlik Karşılığı

İşlem	Bölüm	Kalan
$45 \div 8$	5	5 (en düşük anlamlı basamak)
$5 \div 8$	0	5 (en yüksek anlamlı basamak)

Kalanlar aşağıdan yukarı okunduğunda: $(45)_{10} = (55)_8$. Doğrulama: $5 \cdot 8^1 + 5 \cdot 8^0 = 40 + 5 = 45$. ✓

Örnek 1.13: $(202)_{10}$ 'un Onaltılık Karşılığı: Bir Tutarlılık Sınaması

Bu örneği daha önce, Bölüm 1.2.7'da bit gruplamasıyla da çözüp $(CA)_{16}$ bulacağız. Burada *başka bir yöntemle* aynı sonuca ulaşarak iki yöntemi çapraz doğrulamış olacağız.

İşlem	Bölüm	Kalan
$202 \div 16$	12	10 = A (en düşük anlamlı basamak)
$12 \div 16$	0	12 = C (en yüksek anlamlı basamak)

Kalanlar aşağıdan yukarı okunduğunda: $(202)_{10} = (CA)_{16}$.

Örnek 1.14: $(500)_{10}$ 'un Onaltılık Karşılığı

İşlem	Bölüm	Kalan
$500 \div 16$	31	4 (en düşük anlamlı basamak)
$31 \div 16$	1	15 = F
$1 \div 16$	0	1 (en yüksek anlamlı basamak)

Kalanlar aşağıdan yukarı okunduğunda: $(500)_{10} = (1F4)_{16}$. Doğrulama: $1 \cdot 16^2 + 15 \cdot 16^1 + 4 \cdot 16^0 = 256 + 240 + 4 = 500$. ✓

1.2.7 İkili, Sekizlik ve Onaltılık Arasında Gruplama

İkili-Sekizlik-Onaltılık Arası Gruplama: İkili sayılar bilgisayar için idealdir ama insan gözüyle takip edilmesi zahmetlidir. Sekizlik (octal, $r = 8$) ve onaltılık (hexadecimal, $r = 16$) sistemler tam bu yüzden kullanılır: $8 = 2^3$ ve $16 = 2^4$ olduğu için, ikili ile bu tabanlar arasındaki dönüşüm bir bölme/çarpma *gerektirmez*. Yalnızca bitlerin gruplanmasıyla yapılır: her **3 bit** bir sekizlik basamağa, her **4 bit** bir onaltılık basamağa karşılık gelir.

Örnek 1.15: İkiliden Onaltılığa Gruplama

$(11001010)_2$ sayısını onaltılığa çevirelim; sağdan başlayarak 4'erli gruplarız:

$$\underbrace{1100}_C \underbrace{1010}_A \Rightarrow (CA)_{16}$$

Örnek 1.16: Onaltılıdan İkiliye Açma: Ters Yön

Gruplama tersine de çevrilebilir: her onaltılık basamak, kendi 4-bit ikili karşılığıyla değiştirilir. $(CA)_{16}$ 'yı ikiliye açalım:

$$\underbrace{C}_{1100} \underbrace{A}_{1010} \Rightarrow (11001010)_2$$

Bu, bir önceki örneğin tam tersi — aynı sayı çifti arasında gidip gelebildiğimizi gösterir.

Örnek 1.17: İkiliden Sekizliğe Gruplama

$(101101110)_2$ sayısını sekizliğe çevirelim; bu kez bit sayısı (9) tam olarak 3'e bölündüğü için baştan sıfır eklemeye gerek yok, doğrudan sağdan 3'erli gruplanır:

$$\underbrace{101}_5 \underbrace{101}_5 \underbrace{110}_6 \Rightarrow (556)_8$$

Doğrulama: $5 \cdot 8^2 + 5 \cdot 8^1 + 6 \cdot 8^0 = 320 + 40 + 6 = 366$, ve $(101101110)_2 = 1 \cdot 256 + 1 \cdot 64 + 1 \cdot 32 + 1 \cdot 8 + 1 \cdot 4 + 1 \cdot 2 = 366$. ✓

Alıştırma 1.1 ★ $(11001010)_2$ sayısını, yukarıdaki örnekteki gibi ama bu kez **sekizlik** sisteme, sağdan 3'erli gruplayarak çeviriniz. (İpucu: 8 bit 3'e tam bölünmez; en sola gerektiği kadar 0 ekleyerek grupları tamamlayınız.)

Çözüm: 8 biti 3'e tam bölmek için sola bir 0 eklenir, 9 bit elde edilir; sağdan 3'erli gruplanır:

$$\underbrace{011}_3 \underbrace{001}_1 \underbrace{010}_2 \Rightarrow (312)_8$$

Doğrulama: $(312)_8 = 3 \cdot 8^2 + 1 \cdot 8^1 + 2 \cdot 8^0 = 192 + 8 + 2 = 202$, ve $(11001010)_2 = (202)_{10}$ — iki sonuç örtüşüyor.

1.2.8 Tabanı Bilinmeyen İşlemlerde Taban Bulma

Şimdiye kadar hep taban r 'yi bilip bir sayıyı çeviriyorduk. Bu alt bölümde işi tersine çevireceğiz: pozisyonel sistem tanımındaki ağırlıklı toplam formülünü ($\sum d_i r^i$), tabanın kendisini **bilinmeyen** kabul ederek bir denklem olarak kuracak ve r için çözeceğiz. Tek kısıtlama akılda tutulmalı: bir ifadede kullanılan her basamak, tabandan *küçük* olmak zorundadır ($d_i < r$) — bulunan çözüm bu kısıtla sınanmalıdır.

Örnek 1.18: $(23)_x$ Sayısının Ondalık Karşılığı 19 İse x Nedir?

Ağırlıklı toplam formülü, üsler açık yazılarak kurulur; $x^0 = 1$ olduğu için birler basamağı x ile çarpılmaz, sadece kendisi kalır:

$$(23)_x = 2 \cdot x^1 + 3 \cdot x^0 = 2x + 3$$

Bu ifade 19'a eşitlenip x için çözülür:

$$2x + 3 = 19 \Rightarrow 2x = 16 \Rightarrow x = 8$$

Kısıt kontrolü: kullanılan basamaklar $\{2, 3\}$, $x = 8$ bunların ikisinden de büyük. ✓

Örnek 1.19: $(34)_x + (21)_x = (105)_x$ İse x Nedir?

Üç tarafı da önce üsler açık yazılarak kuralım:

$$(3 \cdot x^1 + 4 \cdot x^0) + (2 \cdot x^1 + 1 \cdot x^0) = 1 \cdot x^2 + 0 \cdot x^1 + 5 \cdot x^0$$

Sadeleştirilince:

$$(3x + 4) + (2x + 1) = x^2 + 5 \Rightarrow 5x + 5 = x^2 + 5 \Rightarrow x^2 - 5x = 0 \Rightarrow x(x - 5) = 0$$

İki kök: $x = 0$ ve $x = 5$. $x = 0$ geçerli bir taban olamaz (ayrıca kısıt $x > 4$ 'ü de sağlamaz); $x = 5$ kısıtı sağlar ($5 > 4$). ✓Doğrulama: $(34)_5=19$, $(21)_5=11$, toplam $30 = (105)_5$.

Örnek 1.20: $(32)_x + (24)_x = (100)_x$ İse x Nedir?

Bu kez RHS üç basamaklı olduğu için denklem *ikinci dereceden* çıkar. Üsler açık:

$$(3 \cdot x^1 + 2 \cdot x^0) + (2 \cdot x^1 + 4 \cdot x^0) = 1 \cdot x^2 + 0 \cdot x^1 + 0 \cdot x^0$$

Sadeleştirilince:

$$(3x + 2) + (2x + 4) = x^2 \Rightarrow x^2 - 5x - 6 = 0 \Rightarrow (x - 6)(x + 1) = 0$$

İki kök: $x = 6$ ve $x = -1$. Negatif taban fiziksel olarak anlamsız olduğundan $x = -1$ reddedilir; $x = 6$ kısıtı da sağlar ($6 > 4$). ✓Doğrulama: $(32)_6=20$, $(24)_6=16$, toplam $36 = (100)_6$.

1.2.9 Kesirli Sayı Dönüşümleri

Pozisyonel gösterim yalnızca tam sayılarla sınırlı değildir; noktadan sonraki basamaklar *negatif* kuvvetler taşır:

$$(0.d_{-1}d_{-2} \dots d_{-m})_r = \sum_{i=1}^m d_{-i} \cdot r^{-i}$$

Tabandan ondalığa dönüşüm yine doğrudan bu toplamla yapılır — ör. $(0.101)_2 = 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = 0.5 + 0 + 0.125 = (0.625)_{10}$. Ondalıktan bir tabana geçiş ise, tam sayı kısmından **farklı ve ters işleyen** bir yöntem gerektirir.

Örnek 1.21: Ondalıktan Bir Tabana Dönüşüm: Ardışık Çarpma Yöntemi

Kesirli kısım hedef tabanla ardışık olarak çarpılır; her adımda taşan tam sayı basamağı sonucun bir hanesi olur ve kalan kesirle işleme devam edilir. $(0.6875)_{10}$ 'u ikiliye çevirelim:

İşlem	Sonuç	Taşan Basamak
0.6875×2	1.375	1 (en yüksek anlamlı bit)
0.375×2	0.75	0
0.75×2	1.5	1
0.5×2	1.0	1 (en düşük anlamlı bit)

Kesir 0'a düştüğü için işlem sonlanır. Taşan basamaklar **yukarıdan aşağı** okunur: $(0.6875)_{10} = (0.1011)_2$.

Bilgi Notu 1.4: Tuzak: Yön Simetrik Değildir

Tam sayı dönüşümünde (ardışık bölme) sonuç *aşağıdan yukarı* okunurken, kesir dönüşümünde (ardışık çarpma) sonuç *yukarıdan aşağı* okunur. Bu asimetri sık yapılan bir hatanın kaynağıdır. Bir sayının tam ve kesirli kısımlarını ayrı ayrı çevirip doğru yönde birleştirmeyi unutmayın.

Bilgi Notu 1.5: Her Kesir Sonlanmaz

Her ondalık kesir, hedef tabanda sonlu basamakla ifade edilemez. Örneğin $(0.1)_{10}$ 'u ikiliye çevirmeye çalışırsanız ardışık çarpma işlemi hiçbir zaman kesri 0'a düşürmez; sonuç sonsuz tekrar eden bir ikili dizidir $(0.000110011 \dots)$. Bu olgu, kayan noktalı sayı gösteriminde karşılaşılan $0.1 + 0.2 \neq 0.3$ gibi tanınmış tuhaflıkların temel nedenidir. Konuya kayan nokta gösterimini işlerken geri döneceğiz.

Alıştırma 1.2 ★ $(0.375)_{10}$ 'u ardışık çarpma yöntemiyle ikiliye çeviriniz.

Çözüm:

İşlem	Sonuç	Taşan Basamak
0.375×2	0.75	0
0.75×2	1.5	1
0.5×2	1.0	1

Yukarıdan aşağı okunduğunda: $(0.375)_{10} = (0.011)_2$. Doğrulama: $0 \cdot 2^{-1} + 1 \cdot 2^{-2} + 1 \cdot 2^{-3} = 0.25 + 0.125 = 0.375$. ✓

Örnek 1.22: Sekizlik Kesir Dönüşümü

Ardışık çarpma yöntemi herhangi bir tabana genelleşir; 8 ile çarpılır. $(0.513)_{10}$ 'u sekizliğe çevirelim:

İşlem	Sonuç	Taşan Basamak
0.513×8	4.104	4
0.104×8	0.832	0
0.832×8	6.656	6
0.656×8	5.248	5
0.248×8	1.984	1
0.984×8	7.872	7

Kesir hiç 0'a düşmüyor (beklendiği gibi — her kesir sonlanmaz); 6 basamak hassasiyetle kesersek: $(0.513)_{10} \approx (0.406517)_8$.

Örnek 1.23: Karma Sayı Dönüşümü: Tam ve Kesirli Kısım Birlikte

Hem tam sayı hem kesir içeren bir sayıyı çevirirken, **iki kısım ayrı ayrı çevrilip sonra birleştirilir** — tam kısım için ardışık bölme, kesirli kısım için ardışık çarpma. $(41.6875)_{10}$ 'u ikiliye çevirelim.

Tam kısım (41), ardışık bölme ile:

İşlem	Bölüm	Kalan
$41 \div 2$	20	1 (en düşük anlamlı bit)
$20 \div 2$	10	0
$10 \div 2$	5	0
$5 \div 2$	2	1
$2 \div 2$	1	0
$1 \div 2$	0	1 (en yüksek anlamlı bit)

Kalanlar aşağıdan yukarı: $(41)_{10} = (101001)_2$.

Kesirli kısım (0.6875), ardışık çarpma ile — bunu daha önce hesaplamıştık: $(0.6875)_{10} = (0.1011)_2$.

İki sonuç birleştirilir:

$$(41.6875)_{10} = (101001.1011)_2$$

1.3 İkili Aritmetik: Toplama, Çıkarma, Çarpma ve Bölme

Şimdiye kadar sayıları temsil etmeyi ve tabanlar arasında çevirmeyi öğrendik. Bu bölümde bu sayılar üzerinde temel aritmetik işlemleri — toplama, çıkarma, çarpma, bölme — nasıl yapacağımızı ele alıyoruz. r tabanındaki aritmetik işlemler, ilkokulda öğrendiğimiz ondalık kurallarla **birebir aynı mantıkla** işler; tek fark, her basamağın 0'dan $r-1$ 'e kadar sınırlı olması ve elde/borç eşiğinin 10 değil r olmasıdır. Bir sonraki bölümde (Tümleyenler) çıkarmayı toplamaya çeviren bir "trik" öğreneceğiz — ama önce, optimize edeceğimiz ham işlemi elle nasıl yaptığımızı bilmemiz gerekiyor. Her işlem için önce genel r -tabanlı matematiksel formülü kuracağız, sonra $r = 2$ özel durumunu (ikili) örneklerle pekiştireceğiz — bu formüller, ileride mantık kapılarıyla gerçek toplayıcı/çarpıcı/bölücü *devreleri* tasarlarken birebir kullanılacak.

1.3.1 İkili Toplama

Genel Toplama Formülü: n basamaklı, r tabanındaki $A = (a_{n-1} \dots a_1 a_0)_r$ ile $B = (b_{n-1} \dots b_1 b_0)_r$ toplanırken, her basamak i için (sağdan sola, $i=0$ 'dan başlayarak):

$$s_i = (a_i + b_i + c_i) \bmod r, \quad c_{i+1} = \left\lfloor \frac{a_i + b_i + c_i}{r} \right\rfloor$$

Burada c_i , bir önceki basamaktan gelen elde ($c_0=0$). $a_i, b_i, c_i \leq r-1$ olduğundan toplamaları en fazla $2(r-1) + 1 = 2r-1 < 2r$ 'dir; bu yüzden c_{i+1} her zaman 0 veya 1'dir. Hiçbir zaman 2 veya daha büyük bir elde oluşmaz.

İkilide ($r=2$) bu formül $s_i = (a_i + b_i + c_i) \bmod 2$, $c_{i+1} = \lfloor (a_i + b_i + c_i)/2 \rfloor$ 'ye indirgenir —

ve bu, ileride kuracağımız bir "tam toplayıcı" (full adder) mantık kapısı devresinin giriş-çıkış ilişkisinin ta kendisidir. Ondalıkta iki basamağın toplamı 10'u geçince elde doğar; ikilide eşik $r=2$ 'dir: $1 + 1 = (10)_2$ – yani bu basamağa 0 yazılır, bir üst basamağa 1 elde gider.

Örnek 1.24: $(101101)_2 + (100111)_2$

Sağdan sola, sütun sütun:

Basamak	İşlem	Yaz	Elde
0	$1 + 1$	0	1
1	$0 + 1 + 1$	0	1
2	$1 + 1 + 1$	1	1
3	$1 + 0 + 1$	0	1
4	$0 + 0 + 1$	1	0
5	$1 + 1 + 0$	0	1

Son elde (1) yeni bir basamak olarak yazılır: $(1010100)_2$. Doğrulama: $(101101)_2 = 45$, $(100111)_2 = 39$, $45 + 39 = 84 = (1010100)_2$. ✓ Bu bit-bit prosedür, donanımda birebir bir *tam toplayıcı* (full adder) devresine dönüşür. Devresini ilerideki Kombinyasyonel Mantık bölümünde kuracağız.

Alıştırma 1.3 ★ $(1101)_2 + (1011)_2$ işlemini sütun sütun (elde takip ederek) yapınız.

Çözüm:

Basamak	İşlem	Yaz	Elde
0	$1 + 1$	0	1
1	$0 + 1 + 1$	0	1
2	$1 + 0 + 1$	0	1
3	$1 + 1 + 1$	1	1

Son elde (1) yeni basamak olur: $(11000)_2$. Doğrulama: $13 + 11 = 24 = (11000)_2$. ✓

1.3.2 İkili Çıkarma

Genel Çıkarma Formülü: n basamaklı $A - B$ işlemi için, her basamak i 'de:

$$d_i = \begin{cases} a_i - k_i - b_i & \text{eğer } a_i - k_i \geq b_i \\ (a_i - k_i - b_i) + r & \text{eğer } a_i - k_i < b_i \end{cases} \quad k_{i+1} = \begin{cases} 0 & \text{eğer } a_i - k_i \geq b_i \\ 1 & \text{eğer } a_i - k_i < b_i \end{cases}$$

Burada k_i , bir önceki basamaktan gelen borç ($k_0=0$). İkinci durumda $(a_i - k_i < b_i)$, mevcut basamağa taban r **eklenerek** (yani "komşudan r birim ödünç alınarak") fark hesaplanır. Ondalıkta bu eklenen değer 10, ikilide 2'dir.

Ondalıkta borç alırken basamağa 10 eklenir; ikilide borç alırken basamağa sadece 2 eklenir. Genel kural: **borcun değeri, her zaman kullanılan tabana eşittir.**

Örnek 1.25: $(101101)_2 - (100111)_2$

Basamak	İşlem	Yaz	Borç?
0	$1 - 1$	0	hayır
1	$0 - 1 \rightarrow (0 + 2) - 1$	1	evet, üstten alındı
2	$(1 - 1) - 1 \rightarrow (0 + 2) - 1$	1	evet
3	$(1 - 1) - 0$	0	hayır
4	$0 - 0$	0	hayır
5	$1 - 1$	0	hayır

Sonuç: $(000110)_2$. Doğrulama: $45 - 39 = 6 = (000110)_2$. ✓

Alıştırma 1.4 ★ $(1101)_2 - (1011)_2$ işlemini sütun sütun (borç takip ederek) yapınız.

Çözüm:

Basamak	İşlem	Yaz	Borç?
0	$1 - 1$	0	hayır
1	$0 - 1 \rightarrow (0 + 2) - 1$	1	evet
2	$(1 - 1) - 0$	0	hayır
3	$(1 - 0) - 1$	0	hayır

Sonuç: $(0010)_2$. Doğrulama: $13 - 11 = 2 = (0010)_2$. ✓

1.3.3 İkili Çarpma

Genel Çarpma Formülü: A (çarpılan) ile n basamaklı $B = (b_{n-1} \dots b_1 b_0)_r$ (çarpan) çarpımı, çarpanın her basamağının ürettiği bir *kısmi çarpımın*, doğru yere kaydırılıp toplanmasıyla bulunur:

$$A \times B = \sum_{i=0}^{n-1} b_i \cdot A \cdot r^i$$

İkilide ($r=2$) çarpan basamakları b_i yalnızca 0 veya 1 olabileceğinden, her terim $(b_i \cdot A \cdot 2^i)$ ya tamamen **sıfır** ya da A 'nın i basamak **sola kaydırılmış bir kopyasıdır**. Çarpma işlemi hiçbir zaman gerçek bir çarpım gerektirmez, sadece kaydırma ve toplama. Bu yüzden ikili çarpmaya *kaydır-ve-topla* (shift-and-add) da denir.

Örnek 1.26: $(1011)_2 \times (101)_2$

Çarpanın basamakları (sağdan sola) $b_0=1$, $b_1=0$, $b_2=1$. Her biri için kısmi çarpımı, **zaten kaydırılmış tam genişliğiyle** yazalım (böylece toplama sıradan basamak-hizalı toplamaya döner):

Terim	Değer	Açıklama
$b_0 \cdot A \cdot 2^0$	001011	A , kaydırmaz
$b_1 \cdot A \cdot 2^1$	000000	$b_1=0$ olduğu için tamamen sıfır
$b_2 \cdot A \cdot 2^2$	101100	A , 2 basamak sola kaydırılmış

Üç terim de artık aynı genişlikte (6 bit); Bölüm 1.3.1'daki sıradan ikili toplama ile eklenir:

$$001011 + 000000 + 101100 = 110111$$

Doğrulama: $(1011)_2 = 11$, $(101)_2 = 5$, $11 \times 5 = 55 = (110111)_2$. ✓

1.3.4 İkili Bölme

Bölme, diğer üç işlemten farklıdır: basamak başına kapalı bir formülle değil, adım adım işleyen bir **algoritma**yla tanımlanır (uzun bölme). Önce ne aradığımızı netleştirelim:

Bölmenin Tanımı: A (bölünen) ve $B \neq 0$ (bölen) verildiğinde, bölme işlemi

$$A = Q \cdot B + R, \quad 0 \leq R < B$$

eşitliğini sağlayan Q (bölüm) ve R (kalan) değerlerini bulmaktır.

Genel Bölme Algoritması (Kaydır-ve-Çıkar): A 'nın basamakları en anlamlıdan (soldan) başlayarak tek tek işlenir. Bir çalışma kaydı R (0'dan başlar) her adımda önce tabana göre kaydırılıp bir sonraki basamak eklenir, sonra bölene karşılaştırılır:

$$R \leftarrow r \cdot R + a_i \quad \text{ardından} \quad \begin{cases} q_i = 1, R \leftarrow R - B & \text{eğer } R \geq B \\ q_i = 0 & \text{eğer } R < B \end{cases}$$

İkilide ($r=2$) bu, her basamakta yalnızca tek bir soruya indirgenir: "**şu anki R , bölenden büyük veya eşit mi?**" Cevap evetse bölüm biti 1 ve B çıkarılır, hayırsa bölüm biti 0. Bu basitlik (kaydır, karşılaştı, gerekirse çıkar), donanımda *ardışık bölücü* devrelerinin temelini oluşturur.

Örnek 1.27: $(1011)_2 \div (11)_2$

$A = 1011$ (11), $B = 11$ (3). Bitler soldan sağa işlenir:

Adım	Getirilen bit	$R \leftarrow 2R + a_i$	$R \geq B$ mi?	Bölüm biti, yeni R
1	1	1	$1 < 3$, hayır	$q=0$, $R=1$
2	0	2	$2 < 3$, hayır	$q=0$, $R=2$
3	1	5	$5 \geq 3$, evet	$q=1$, $R=5-3=2$
4	1	5	$5 \geq 3$, evet	$q=1$, $R=5-3=2$

Bölüm bitleri sırayla 0, 0, 1, 1: $Q = (0011)_2 = 3$. Son $R = 2 = (10)_2$, kalan. Doğrulama:

$$11 = 3 \times 3 + 2. \checkmark$$

Alıştırma 1.5 ★★ $(1110)_2 \div (11)_2$ işlemini yukarıdaki tabloyla aynı yöntemle yapınız.

Çözüm: $A = 1110$ (14), $B = 11$ (3):

Adım	Getirilen bit	$R \leftarrow 2R + a_i$	$R \geq B$ mi?	Bölüm biti, yeni R
1	1	1	$1 < 3$, hayır	$q=0, R=1$
2	1	3	$3 \geq 3$, evet	$q=1, R=3-3=0$
3	1	1	$1 < 3$, hayır	$q=0, R=1$
4	0	2	$2 < 3$, hayır	$q=0, R=2$

$Q = (0100)_2 = 4$, kalan $R = (10)_2 = 2$. Doğrulama: $14 = 4 \times 3 + 2. \checkmark$

1.4 Tümleyenler

Şimdiye kadar sayıları bir tabandan diğerine çevirdik. Bu bölümde farklı bir amaca hizmet eden bir araç kuracağız: *tümleyenler*.

Bilgi Notu 1.6: Neden Çıkarmayı Toplamaya Çeviriyoruz?

Bir bilgisayarın işlemcisi, aslında milyonlarca küçük anahtardan (transistörden) örülmüş bir ağıdır. Bu transistörler, ikili girişlerden ikili çıkış üreten *mantık kapılarına* (AND, OR, NOT vb.) – Bölüm 2’de göreceğimiz kapılara – gruplanır, kapılar da bir araya gelerek *toplayıcı* (adder) gibi aritmetik devreleri oluşturur. Sorun şu: toplama işlemini yapan bir devre ile çıkarma işlemini yapan bir devre, **birbirinden farklı** kapı düzenlemeleri gerektirir – yani ikisini de donanımda gerçeklemek isterseniz iki ayrı devre kurmanız, dolayısıyla iki katı transistör harcamanız gerekir. Ama eğer $M - N$ işlemini $M + (\text{tümleyen}(N))$ olarak yeniden yazabilirsek, çıkarma işlemi de aynı toplayıcı devre üzerinden yürür. Ayrı bir ”çıkarcı” devre tasarlamaya hiç gerek kalmaz. Tümleyenlerin donanım tasarımındaki asıl değeri budur: transistör/kapı sayısını neredeyse yarıya indiren köklü bir basitleştirme. Bu devrenin somut kapı şemasını, toplayıcı devreleri işlediğimizde (ilerideki Toplayıcı Devreler bölümünde) göreceğiz.

1.4.1 Genel Tümleyen Tanımı

Tümleyen Türleri: n basamaklı, r tabanında bir N sayısı için iki tür tümleyen tanımlanır:

- **$(r-1)$ 'in tümleyeni** (diminished radix complement – ikilide ”1’in tümleyeni”, ondalıkta ”9’un tümleyeni”):

$$(r-1)\text{'in tümleyeni} = (r^n - 1) - N$$

- **r 'nin tümleyeni** (radix complement – ikilide ”2’nin tümleyeni”, ondalıkta ”10’un tümleyeni”):

$$r\text{'nin tümleyeni} = r^n - N = [(r-1)\text{'in tümleyeni}] + 1$$

İkinci tanımın ilk tanıma baęlı olması pratik aıdan nemlidir: $(r-1)$ 'in tmleyenini hesaplamak ok kolaydır (ařaęıda greceęiz), r 'nin tmleyeni de ona sadece 1 eklenerek bulunur.

1.4.2 Ondalık rneklerle Tmleyen Hesabı

Kavramı nce ařına olduęumuz ondalık sistemde ($r=10$) somutlařtırılm. $(r-1)$ 'in tmleyenini, yani 9'un tmleyenini, hesaplamamızın pratik yolu: **her basamaęı 9'dan ıkarmaktır** – nk $(10^n - 1)$ sayısı n tane 9'dan oluřur ($999 \dots 9$), ve basamak basamak ıkarma hibir bor gerektirmez.

rnek 1.28: 546700'n 9'un ve 10'un Tmleyeni

6 basamaklı 546700 sayısının 9'un tmleyeni, her basamaęı 9'dan ıkararak:

$$9-5, 9-4, 9-6, 9-7, 9-0, 9-0 \Rightarrow 453299$$

Doęrulama: $(10^6 - 1) - 546700 = 999999 - 546700 = 453299. \checkmark$

10'un tmleyeni, 9'un tmleyenine 1 eklenerek bulunur:

$$453299 + 1 = 453300$$

Doęrulama: $10^6 - 546700 = 453300. \checkmark$

rnek 1.29: 012398'in 9'un Tmleyeni

Bařtaki sıfır da bir basamaktır, aynı kurala tabidir:

$$9-0, 9-1, 9-2, 9-3, 9-9, 9-8 \Rightarrow 987601$$

1.4.3 Kısayol Yntemi ve İspatı

Bilgi Notu 1.7: Pratik Kısayol: r 'nin Tmleyenini Doęrudan Almak

r 'nin tmleyenini iki adımda (nce $(r-1)$ 'in tmleyenini alıp sonra 1 eklemek yerine) **tek adımda** bulmanın bir kısayolu vardır: saędan bařlayarak **sondaki sıfırları ve ilk sıfır-olmayan basamaęı olduęu gibi bırakın**, geri kalan (soldaki) basamakları $(r-1)$ 'den ıkartın.

rnek 1.30: Kısayolla 246700'n 10'un Tmleyeni

Sondaki iki 0 aynı kalır; ilk sıfır olmayan basamak (7) 10'dan ıkarılır ($10-7 = 3$); geri kalan basamaklar (2, 4, 6, soldan saęa) 9'dan ıkarılır (7, 5, 3):

$$246700 \Rightarrow 753300$$

Doęrulama: $10^6 - 246700 = 753300. \checkmark$

Bilgi Notu 1.8: Kısayol Neden İşliyor? – Borç Yayılımı İspatı

r 'nin tümleyeni, r^n (yani n basamaklı $\underbrace{1\ 00\ \dots\ 0}_n$ sayısı) ile N arasındaki farktır: $r^n - N$.

Bu çıkarmayı elle, sağdan sola borç alarak yapalım. N 'nin sondan itibaren k tane sıfırı olsun. İlk k basamakta $0 - 0$ işlemi yapılır. Borca hiç gerek yoktur, sonuç 0 kalır. Basamak k 'de (sondan itibaren ilk sıfır-olmayan basamak, değeri d) işlem $0 - d$ 'dir; bu borç gerektirir, sonuçta $(r - d)$ elde edilir ve bir borç sola – basamak $k+1$ 'e – yayılır. Basamak $k+1$ 'de r^n 'nin kendi basamağı da 0 olduğundan, gelen borç yüzünden işlem aslında $(0 - 1) - d_{k+1} = (r - 1) - d_{k+1}$ 'dir (yine bir borç daha sola yayılarak) – ve bu, en soldaki basamağa kadar devam eder. Yani ilk sıfır-olmayan basamağın solundaki **her** basamak, gelen borç nedeniyle otomatik olarak $(r-1)$ 'den çıkarılmış olur. Kısayol, bu borç yayılımının doğrudan bir sonucudur.

1.4.4 İkilide Tümleyenler

Aynı tanımlar $r = 2$ için doğrudan uygulanır. $(r-1) = 1$ olduğundan, **1'in tümleyeni, her biti ters çevirmektir** (NOT) – çünkü $1 - d_i$ işlemi tam olarak biti tersine çevirir ($1 - 0 = 1$, $1 - 1 = 0$). **2'nin tümleyeni** ise 1'in tümleyenine 1 eklenerek bulunur.

Örnek 1.31: $(1011000)_2$ 'nin 1'in ve 2'nin Tümleyeni

Her bit ters çevrilir:

$$(1011000)_2 \xrightarrow{\text{NOT}} (0100111)_2 \quad (1'\text{'in tümleyeni})$$

Bu sonuca 1 eklenir:

$$0100111 + 1 = 0101000 \quad (2'\text{'in tümleyeni})$$

Örnek 1.32: Kısayolla $(1011000)_2$ 'nin 2'nin Tümleyeni

Sağdan başlayarak sondaki sıfırları (000) ve ilk 1'i olduğu gibi bırakırız (...1000), geri kalan sol taraf (101) ters çevrilir (010):

$$(1011000)_2 \Rightarrow (0101000)_2$$

Yukarıdaki uzun yöntemle bulduğumuz sonuçla aynı. ✓ Bu kısayol, elle hesap yaparken (ör. bir sonraki bölümdeki çıkarma örneklerinde) ciddi zaman kazandırır.

1.4.5 Sekizlik ve Onaltılıkta $(r-1)$ 'in Tümleyeni

Aynı mantık her tabana genelleşir: sekizlikte $(r-1) = 7$ olduğundan, $(r-1)$ 'in tümleyeni **her basamağı 7'den çıkararak** bulunur; onaltılıkta $(r-1) = 15 = F$ olduğundan, **her basamak F'den çıkarılır** (harf basamaklarını önce Tablo 1.1 ile sayıya çevirmeyi, sonucu da gerekirse tekrar harfe çevirmeyi unutmayın).

rnek 1.33: $(5643)_8$ 'n 7'nin Tmleyeni

Her basamak 7'den ıkarılır:

$$7-5, 7-6, 7-4, 7-3 \Rightarrow 2134$$

Yani $(5643)_8$ 'n 7'nin tmleyeni $(2134)_8$ 'dir. Doęrulama: $(7777)_8 - (5643)_8$ ondalıkta $4095 - 2979 = 1116$, ve 1116'nın sekizlik karřılıęı $(2134)_8$ 'tr. ✓

rnek 1.34: $(3A9)_{16}$ 'un F 'nin Tmleyeni

Her basamak (harfler nce sayıya evrilerek) $F(= 15)$ 'ten ıkarılır: $15-3 = 12 = C$, $15-10 = A \rightarrow 5$, $15-9 = 6$:

$$(3A9)_{16} \Rightarrow (C56)_{16}$$

Doęrulama: $(FFF)_{16} - (3A9)_{16}$ ondalıkta $4095 - 937 = 3158$, ve 3158'in onaltılık karřılıęı $(C56)_{16}$ 'dir. ✓

1.4.6 Kesirli (Noktalı) Sayılarda Tmleyen Alma

Buraya kadarki tm tanımlar, sayının bir radix noktası (ondalık/ikili nokta) iermedięini varsaydı. Eęer N 'nin bir noktası varsa, tmleyeni almadan nce nokta **geici olarak kaldırılır**, sayı sanki tam sayıymıř gibi tmleyeni alınır, ve nokta **aynı grelı konumuna geri konur**.

rnek 1.35: (546.700) 'n 9'un Tmleyeni

Nokta geici olarak kaldırılır: 546700. Bunun 9'un tmleyeni (daha nce hesaplamıřtık): 453299. Nokta, sondan c basamaęın nndeki aynı grelı konumuna geri konur:

$$(546.700) \Rightarrow (453.299)$$

rnek 1.36: $(101.11)_2$ 'nin 2'nin Tmleyeni

Nokta kaldırılır: 10111 (5 basamak). 2'nin tmleyeni: 1'in tmleyeni 01000, $+1 = 01001$. Nokta, sondan iki basamaęın nndeki aynı grelı konuma geri konur:

$$(101.11)_2 \Rightarrow (010.01)_2$$

Yntem tam sayılarla tamamen aynıdır. Tek fark, iřlem bitince noktanın eski yerine konmasıdır.

Tmleyenin Tmleyeni: r 'nin tmleyeni tanımı gereęi $r^n - N$ olduęundan, bu sonucun bir kez daha r 'nin tmleyenini almak

$$r^n - (r^n - N) = N$$

verir – yani **tmleyenin tmleyeni, sayının kendisini geri verir**. Bu, ilerideki ıkarma prosedrnde "sonu negatifse, bulduęunuz deęerin bir kez daha tmleyenini alıp bařına eksi koyun" adımıının neden doęru sonucu verdięini aıklar: negatif durumda

elde ettiğimiz ara sonuç, aslında gerçek büyüklüğün tümleyenidir; tümleyenini tekrar almak bizi gerçek büyüklüğe geri götürür.

1.4.7 r 'nin Tümleyeniyile Çıkarma İşlemi

Tümleyenlerin asıl amacına gelelim: $M - N$ çıkarma işlemini toplamaya çevirmek. n basamaklı M ve N için kural şöyledir:

r 'nin Tümleyeniyile Çıkarma Kuralı:

1. $M + [r\text{'nin tümleyeni}(N)]$ toplamını hesaplayın.
2. Sonuç n basamaktan **taşarsa** (bir "uç elde" oluşursa), taşan basamağı **atın**. Kalan n basamak doğrudan $M - N$ 'dir ve sonuç **pozitif** ($M \geq N$ demektir).
3. Sonuç n basamaktan **taşmazsa**, elde ettiğiniz n basamaklı sonuç aslında $N - M$ 'nin r 'nin tümleyenidir ($M < N$ demektir). Gerçek büyüklüğü bulmak için bu sonucun bir kez daha r 'nin tümleyenini alın ve başına eksi işareti koyun.

Bilgi Notu 1.9: Önce Basamak Sayılarını Eşitleyin

Bu kural, M ve N 'nin **aynı sayıda basamağa** (n) sahip olmasını varsayar. Tümleyen tanımı da ($r^n - N$) zaten belirli bir n 'ye göre yapılır. Sayıların basamak sayıları farklıysa, kısa olanın **başına sıfır eklenerek** uzun olanla eşitlenir. Aşağıdaki ilk örnekte $N = 3250$ 'nin 4 basamağı var ama $M = 72532$ 'nin 5; bu yüzden N 'yi 03250 olarak yazıp $n = 5$ üzerinden işlem yapıyoruz.

Örnek 1.37: Ondalıkta Çıkarma: Taşma Var, Pozitif Sonuç

$72532 - 03250$ işlemini ($n = 5$ basamak) tümleyenle yapalım. $N = 03250$ 'nin 10'un tümleyeni: $100000 - 03250 = 96750$.

$$M + \text{tümleyen}(N) = 72532 + 96750 = 169282$$

Bu, 5 basamaklı sistemde bir "uç elde" (taşan 1) üretir. Taşan basamak atılır:

$$72532 - 3250 = 69282$$

Doğrulama: $72532 - 3250 = 69282$. ✓

Örnek 1.38: Ondalıkta Çıkarma: Taşma Yok, Negatif Sonuç

Aynı sayılarla ters yönde, $03250 - 72532$ işlemini yapalım. $N = 72532$ 'nin 10'un tümleyeni: $100000 - 72532 = 27468$.

$$M + \text{tümleyen}(N) = 03250 + 27468 = 30718$$

Bu 5 basamađa sığıyor, u elde **yok**. Demek ki gerek sonu negatif: 30718'in 10'un tmleyenini alıp bařına eksi koyuyoruz:

$$100000 - 30718 = 69282 \Rightarrow \text{sonu} = -69282$$

Dođrulama: $3250 - 72532 = -69282$. ✓

rnek 1.39: İkilide ıkarma: 2'nin Tmleyeniyle, Tařma Var

8 bit zerinde $(01010100)_2=84$ 'ten $(01000011)_2=67$ 'yi ıkaralım. N 'nin 2'nin tmleyeni (kısayolla): 10111101.

$$01010100 + 10111101 = \underset{\text{u elde}}{\underline{1}} 00010001$$

U elde atılır: sonu $(00010001)_2 = 17$. Dođrulama: $84 - 67 = 17$. ✓

rnek 1.40: İkilide ıkarma: 2'nin Tmleyeniyle, Tařma Yok

Ters ynde, $67 - 84$ iřlemine yapalım. $84 = (01010100)_2$ 'nin 2'nin tmleyeni: 10101100.

$$01000011 + 10101100 = 11101111$$

8 bite sığıyor, u elde yok. Sonu negatif. $(11101111)_2$ 'nin 2'nin tmleyeni: 00010001 = 17. Sonu: -17 . Dođrulama: $67 - 84 = -17$. ✓

1.4.8 $(r-1)$ 'in Tmleyeniyle ıkarma: Utan-Dolařan Elde

$(r-1)$ 'in tmleyeni (9'un veya 1'in tmleyeni) ile ıkarma yapmak isterseniz, kural bir noktada farklılařır: u elde oluřtuđunda bu sefer **atılmaz**, bunun yerine sonucun en dřk anlamlı basamađına **eklenir**. Buna *utan-dolařan elde* (end-around carry) denir. Bunun nedeni, $(r-1)$ 'in tmleyeninin r 'nin tmleyeninden tam 1 eksik olması: kaybolan bu farkı telafi etmek iin tařan biti geri "dolařtırıyoruz".

rnek 1.41: Ondalıkta 9'un Tmleyeniyle ıkarma

Aynı $72532 - 3250$ iřlemi, bu kez 9'un tmleyeniyle. $N = 03250$ 'nin 9'un tmleyeni: 96749.

$$M + \text{tmleyen}(N) = 72532 + 96749 = 169281$$

U elde (1) atılmaz, en dřk basamađa eklenir:

$$69281 + 1 = 69282$$

Aynı sonu (r 'nin tmleyeni yntemiyle bulduđumuzla zdeř). ✓

Örnek 1.42: İkilide 1'in Tümleyeniyle Çıkarma

84 – 67 işlemini bu kez 1'in tümleyeniyle yapalım. $67 = (01000011)_2$ 'nin 1'in tümleyeni: 10111100.

$$01010100 + 10111100 = \underset{\text{uç elde}}{1} \quad 00010000$$

Uç elde atılmaz, en düşük basamağa eklenir: $00010000 + 1 = 00010001 = 17$. Doğrulama: $84 - 67 = 17$. ✓

Alıştırma 1.6 ★★ 67 – 84 işlemini 8 bit üzerinde **1'in tümleyeni** yöntemiyle (uçtan-dolaşan elde kuralına dikkat ederek) çözünüz.

Çözüm: $84 = (01010100)_2$ 'nin 1'in tümleyeni: 10101011.

$$01000011 + 10101011 = 11101110$$

8 bite sığıyor, uç elde **yok**. Bu durumda uçtan-dolaşan elde kuralı devreye girmez, sonuç doğrudan negatiftir. $(11101110)_2$ 'nin 1'in tümleyeni: $00010001 = 17$. Sonuç: -17 . Doğrulama: $67 - 84 = -17$. ✓

1.5 İşaretili İkili Sayılar

Şimdiye kadar gördüğümüz her sayı *işaretsizdi* — yalnızca sıfır ve pozitif değerleri temsil ediyordu. Ama gerçek dünyada negatif sayılara da ihtiyacımız var. Sıradan aritmetikte bunu bir “–” sembolüyle çözeriz; donanımda ise elimizde yalnızca bitler var, sembol diye bir şey yok. Bu bölümde, bir önceki bölümde kurduğumuz tümleyen aracını kullanarak, negatif sayıları saf bitlerle nasıl temsil edeceğimizi göreceğiz.

1.5.1 İşaret Biti ve Yorumlama

İşaret Biti: İşaretili bir ikili sayıda, en soldaki bit *işaret bitidir*: 0 pozitif, 1 negatif anlamına gelir. Kalan bitler büyüklüğü (ya da tümleyen sistemindeyse, aşağıda göreceğimiz gibi farklı bir şeyi) temsil eder.

Bilgi Notu 1.10: Aynı Bitler, Farklı Anlam

Bir bit dizisinin işaretili mi işaretsiz mi olduğu, dizinin kendisinde yazılı değildir — **yorumlayan tarafından bilinmesi gerekir**. Örneğin 01001: işaretsiz okunursa 9, işaretili okunursa +9'dur (soldaki bit 0 olduğu için ikisi de aynı büyüklüğü verir). Ama 11001: işaretsiz okunursa 25, işaretili okunursa –9'dur. En soldaki 1 işaret biti olarak yorumlandığında, kalan 1001 (9) büyüklüğünün negatifini gösterir.

1.5.2 İşaret-Büyüklük Gösterimi

En sezgisel yöntem, sıradan aritmetikle birebir aynıdır: işaret biti ayrı, büyüklük ayrı saklanır.

Örnek 1.43: -5'in 8 Bitte İşaret-Büyüklik Gösterimi

+5 = 00000101. İşaret-büyüklikte -5, sadece işaret bitini 1 yaparak elde edilir:

$$-5 = 10000101$$

1.5.3 İşaretili Tümleyen Sistemi: 1'in ve 2'nin Tümleyeni

İşaret-büyüklik sezgisel olsa da, donanımda kullanılan asıl yöntem farklıdır: negatif sayı, pozitif sayının **tümleyeni** alınarak elde edilir — işaret biti dahil, **sayının tamamının** tümleyeni.

Örnek 1.44: -9'un 8 Bitte Üç Gösterimi

+9 = 00001001 (8 bite tamamlamak için işaret bitinden sonra gerektiği kadar 0 eklenir).

Gösterim	-9
İşaret-büyüklik (sadece işaret biti değişir)	10001001
İşaretili 1'in tümleyeni (tüm bitler ters çevrilir)	11110110
İşaretili 2'nin tümleyeni (1'in tümleyenine +1)	11110111

Bilgi Notu 1.11: İpucu: Ağırlıklı Toplamla Doğrudan Değer Okuma

2'nin tümleyeni gösterimindeki bir sayının değerini bulmak için her zaman "ters çevir, 1 ekle, başına eksi koy" yapmak gerekmez. Doğrudan bir kısayol vardır: n -bitlik sayı $b_{n-1} \dots b_1 b_0$ için, **en soldaki (işaret) bit negatif ağırlıklı** sayılırsa, değer doğrudan ağırlıklı toplamla okunur:

$$V = -b_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i$$

Örnek 1.45: Kısayolla $(11011)_2$ 'nin Değerinin Bulunması

İşaret biti ($b_4 = 1$) negatif ağırlıklı sayılır:

$$-1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = -16 + 8 + 0 + 2 + 1 = -5$$

Yani $(11011)_2 = -5$ (2'nin tümleyeni gösteriminde).

Bu kısayolun neden çalıştığını görelim. Bölüm 1.4'de r 'nin tümleyenini $r^n - N$ olarak tanımlamıştık. n -bit 2'nin tümleyeninde negatif bir V değeri, bit kalıbı $X = 2^n + V$ olan işaretsiz bir sayı olarak saklanır (yani $V = X - 2^n$). Bit kalıbının işaretsiz değeri $X = \sum_{i=0}^{n-1} b_i 2^i$ 'dir; işaret biti $b_{n-1} = 1$ olduğundan $X = 2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i$ 'dir. Buradan:

$$V = X - 2^n = 2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i - 2^n$$

$$= -2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i$$

Bu, tam olarak yukarıdaki kısayol formülüdür.

Örnek 1.46: Kısayolun Doğrulanması: -9 'un 2 'nin Tümleyeni Gösterimi

Az önce -9 'un 8-bit 2 'nin tümleyeni gösterimini 11110111 olarak bulmuştuk. Aynı kısayolu bu bit kalıbına uygulayalım:

$$\begin{aligned} & -1 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\ & = -128 + 64 + 32 + 16 + 0 + 4 + 2 + 1 = -9 \end{aligned}$$

Doğrulandı.

Alıştırma 1.7 ★ Kısayolu kullanarak $(10101)_2$ 'nin (5-bit 2 'nin tümleyeni) değerini bulunuz.

Çözüm: $-1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = -16 + 0 + 4 + 0 + 1 = -11$.

1.5.4 Neden 2 'nin Tümleyeni Tercih Edilir

Tablo 1.2: 4-Bit İşaretili Sayılar: Üç Gösterimin Karşılaştırması

Ondalık	2 'nin Tümleyeni	1 'in Tümleyeni	İşaret-Büyüklik
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000
-0	—	1111	1000
-1	1111	1110	1001
-2	1110	1101	1010
-3	1101	1100	1011
-4	1100	1011	1100
-5	1011	1010	1101
-6	1010	1001	1110
-7	1001	1000	1111
-8	1000	—	—

Tablo 1.2'ten iki gözlem çıkar:

Bilgi Notu 1.12: Sebep 1: Tek Sıfır

İşaret-büyüklik ve 1 'in tümleyeninde **hem** +0 **hem** -0 var. 16 kombinasyonun biri israf ediliyor, ayrıca "sıfır mı değil mi" karşılaştırması iki farklı bit dizisini kontrol etmeyi ge-

rektiriyor. 2'nin tümleyeninde sıfırın **tek** bir gösterimi var; bu yüzden bir negatif değer daha (-8) sığdırılabiliyor.

Bilgi Notu 1.13: Sebep 2 (Asıl Önemli Olan): Aritmetik Kolaylığı

İşaret-büyüklikte iki sayıyı toplamak için önce işaretlerini **karşılaştırmanız** gerekir: işaretler aynıysa büyüklükleri toplayıp ortak işareti verirsiniz; işaretler farklıysa küçük büyüklüğü büyüktен **çıkartıp** büyük olanın işaretini verirsiniz. Bu, ayrı bir karşılaştırma ve çıkarma devresi gerektirir. 2'nin tümleyeninde ise kural tek cümledir: **işaret biti dahil her şeyi normal topla, işaret basamağından taşan biti at**. Karşılaştırma yok, özel durum yok. Bir sonraki iki alt bölümde bunu somutlaştıracamız. Bu yüzden neredeyse tüm bilgisayarların aritmetik birimleri 2'nin tümleyeni sistemini kullanır.

1.5.5 İşaretlili Toplama

İşaretlili Toplama Kuralı (2'nin Tümleyenini): İki işaretlili sayı, **işaret bitleri dahil** sıradan ikili toplama ile toplanır. İşaret basamağından taşan bir uç elde oluşursa, **atılır**.

Örnek 1.47: Dört Durum, 8 Bit Üzerinde

Aşağıdaki tabloda negatif sayılar (-6 ve -13) zaten kendi **2'nin tümleyenini** gösterimleriyle verilmiştir — yani 11111010, -6 'nın bit dizisidir (bir önceki alt bölümde gördüğümüz yöntemle hesaplanmıştır), rastgele bir dizi değildir. Yeni başlayan biri için bu netleştirme önemlidir: tabloya bakarken "bu 1'lerle dolu dizi nereden çıktı" diye düşünmeyin. Her zaman önce pozitif sayının ikili karşılığını yazıp sonra (gerekliyse) 2'nin tümleyenini alarak elde edilirler.

İşlem	Toplam	Sonuç
$(+6) + (+13)$: 00000110 + 00001101	00010011	+19
$(-6) + (+13)$: 11111010 + 00001101	1 00000111 (uç elde atılır)	+7
$(+6) + (-13)$: 00000110 + 11110011	11111001	-7
$(-6) + (-13)$: 11111010 + 11110011	1 11101101 (uç elde atılır)	-19

Her dört durumda da kural aynı: işaret dahil topla, taşarsa at. Sonucun negatif mi pozitif mi olduğu, sonucun kendi işaret bitinden (0/1) doğrudan okunur.

Bilgi Notu 1.14: Taşma (Overflow)

n bitlik iki sayının toplamı, n bite sığmayan (yani $n+1$ bit gerektiren) bir sonuç üretirse **taşma** oluşur. Kağıt üzerinde sorun değildir — sınırsız genişletebilirsiniz — ama donanımda bit genişliği sabittir, taşan sonuç kayba uğrar. Örneğin 4 bitlik (aralık -8 ile $+7$) bir sistemde $(+5) + (+4) = 0101 + 0100 = 1001$ işlemini yapalım: sonuç 1001, işaret biti 1 olduğu için *negatif* okunur — 2'nin tümleyeninde $1001 = -7$. Ama $5 + 4 = 9$ pozitif

olmalıydı! Sonuç 4 bite sığmadığı ($9 > 7$, aralığın üst sınırını aştığı) için taşma oluştu ve anlamsız bir işaret üretti. Taşmanın devrede nasıl otomatik tespit edildiğini, toplayıcı devresini kurduğumuzda göreceğiz.

Alıştırma 1.8 ★ 4 bit üzerinde $(+3) + (-5)$ işlemini 2'nin tümleyeni kuralıyla yapınız.

Çözüm: $+3 = 0011$. -5 'in 2'nin tümleyeni: $+5 = 0101$, 1'in tümleyeni 1010 , $+1 = 1011$.

$$0011 + 1011 = 1110$$

4 bite sığıyor, uç elde yok. İşaret biti 1, yani negatif: 1110 'nin 2'nin tümleyeni $0010 = 2$, sonuç -2 . Doğrulama: $3 - 5 = -2$. ✓

Alıştırma 1.9 ★★ 4 bit üzerinde $(-3) + (+5)$ işlemini yapınız. Bu kez sonuçta gerçek bir **uç elde** oluşacak, atılması gerekecek.

Çözüm: -3 'ün 2'nin tümleyeni: $+3 = 0011$, 1'in tümleyeni 1100 , $+1 = 1101$. $+5 = 0101$.

$$1101 + 0101 = 10010$$

Bu 5 bit. 4 bitlik sistemde bir **uç elde** (1) oluştu, atılır. Kalan $0010 = 2$. İşaret biti 0, yani pozitif: sonuç $+2$. Doğrulama: $-3 + 5 = 2$. ✓

1.5.6 İşaretli Çıkarma

İşaretli Çıkarma Kuralı (2'nin Tümleyeni): Çıkanın (çıkartılacak sayının) **2'nin tümleyeni**ni (işaret biti dahil) alın ve eksilene **toplayın**. İşaret basamağından taşan uç elde atılır.

Bu kural, çıkarmayı toplamaya indirger: $(\pm A) - (+B) = (\pm A) + (-B)$ ve $(\pm A) - (-B) = (\pm A) + (+B)$. Bir pozitif sayıyı negatife çevirmek (ya da tam tersi) zaten 2'nin tümleyeni almakta ibaret olduğu için, çıkarma her zaman "tümleyen al, topla" işlemine dönüşür.

Örnek 1.48: $(-6) - (-13)$, 8 Bit Üzerinde

$-6 = 11111010$, $-13 = 11110011$. Çıkanın (-13) 2'nin tümleyeni alınır (bu, $+13$ 'ü verir): 00001101 . Eksilene (-6) eklenir:

$$11111010 + 00001101 = 1\ 00000111$$

Uç elde atılır: sonuç $00000111 = +7$. Doğrulama: $(-6) - (-13) = 7$. ✓

Örnek 1.49: $(+9) - (+13)$, 8 Bit Üzerinde: Uç Eldesiz Durum

$+9 = 00001001$, $+13 = 00001101$. Çıkanın $(+13)$ 2'nin tümleyeni alınır (bu, -13 'ü verir): 11110011 . Eksilene $(+9)$ eklenir:

$$00001001 + 11110011 = 11111100$$

Bu kez 8 bite sığıyor, uç elde **yok** — ama bu, bir önceki bölümdeki işaretsiz tümleyen

çıkarmasından farklı olarak burada bir sorun **değildir**: işaretli çıkarmada kural her zaman aynıdır (varsa uç eldeyi at), uç eldenin olup olmaması sonucun işaretini etkilemez, sonuç zaten kendi işaret bitiyle doğru okunur. 11111100'ün işaret biti 1, yani negatif; 2'nin tümleyeni 00000100 = 4, sonuç -4. Doğrulama: $9 - 13 = -4$. ✓

Alıştırma 1.10 ★ (+15) - (+6) işlemini 8 bit üzerinde 2'nin tümleyeni kuralıyla yapınız.

Çözüm: +15 = 00001111. Çıkanın (+6) 2'nin tümleyeni: +6 = 00000110, 1'in tümleyeni 11111001, +1 = 11111010 (-6). Eksilene eklenir:

$$00001111 + 11111010 = 100001001$$

Uç elde atılır: sonuç 00001001 = +9. Doğrulama: $15 - 6 = 9$. ✓

Alıştırma 1.11 ★★ (+4) - (+9) işlemini 8 bit üzerinde 2'nin tümleyeni kuralıyla yapınız.

Çözüm: +4 = 00000100. Çıkanın (+9) 2'nin tümleyeni: +9 = 00001001, 1'in tümleyeni 11110110, +1 = 11110111 (-9). Eksilene eklenir:

$$00000100 + 11110111 = 11110111$$

8 bite sığıyor, uç elde yok - önceki örnekte gördüğümüz gibi bu bir sorun değil, kural aynı şekilde işliyor. İşaret biti 1, yani negatif: 11110111'in 2'nin tümleyeni 00000101 = 5, sonuç -5. Doğrulama: $4 - 9 = -5$. ✓

Bilgi Notu 1.15: Büyük Resim: Tek Bir Devre Yeter

İşaretli sayılar, işaretsiz sayılarla **birebir aynı toplama donanımını** kullanır. Çıkarma bile aynı toplayıcı üzerinden, sadece bir tümleyen alma adımıyla yürür. Bölüm 1.4'in başında sorduğumuz soruya ("neden çıkarmayı toplamaya çeviriyoruz?") artık tam cevabımız var: hem işaretsiz hem işaretli aritmetik, aynı tek toplayıcı devre üzerinden, iki ayrı devre kurmaya hiç gerek kalmadan yürüyor. Bu, neredeyse tüm bilgisayarların aritmetik birimlerinde 2'nin tümleyeni sisteminin evrensel olarak kullanılmasının temel nedenidir.

1.6 IEEE 754 ile Kayan Noktalı Sayı Gösterimi

Bölüm 1.2.9'da, bazı kesirli sayıların ikilide hiçbir zaman sonlanmadığını görmüş, bu durumun kayan noktalı gösterimle olan ilişkisini ise ileriki bir bölüme bırakmıştık. Şimdi sırası geldi: o zaman bahsettiğimiz konuya, kayan noktalı sayı gösterimine, bu bölümde değineceğiz.

Kayan noktalı gösterimin temelinde üç kavram vardır: bir **işaret**, normalize edilmiş bir **anlamli kısım** (mantissa) ve ölçeği belirleyen bir **üs**. Bu üç kavramdan en alışık olmadığımız kısım üstür, çünkü IEEE 754 standardında üs, ham (işaretli) hâliyle değil, *bias* (kaydırma) adı verilen sabit bir değer eklenerek saklanır: gerçek üse bias eklenip elde edilen yeni değere *biaslı üs* denir, ve bit dizisinde fiilen saklanan değer budur, gerçek üs değil. Örneğin tek duyarlıklılı formatta bias 127'dir; gerçek üs -1 ise, saklanan biaslı üs $-1 + 127 = 126$ olur. Bu tercihin mühendislik gerekçesini Bölüm 1.6.3'te göreceğiz; şimdilik terimin ne anlama geldiğini bilmemiz yeterlidir.

1.6.1 Neden Sabit Nokta Yetmez?

Bölüm 1.2.9’da öğrendiğimiz kesirli gösterimde, ikili noktanın konumu **sabittir**. Örneğin 8 bitli tam sayı kısmına, 8 bitli kesir kısmına ayırdığımızı düşünelim: bu düzende hem çok büyük bir sayıyı (milyonlarca gibi) hem çok küçük, hassas bir kesri (milyonda bir gibi) aynı anda temsil edemeyiz. Tam kısma ayrılan 8 bit en fazla 255’e kadar sayı tutabilir, kesir kısmına ayrılan 8 bit de belirli bir hassasiyetin ötesine hiç geçemez. Yani sabit noktalı gösterimde **aralık ile hassasiyet arasında sabit, değiştirilemez bir ödünleşim** vardır.

Kayan noktalı gösterim bu sorunu, bilimsel gösterimden ödünç alınan bir fikirle çözer: çok büyük bir sayıyı 1.234×10^5 olarak, çok küçük bir sayıyı da 6.02×10^{-23} olarak yazabildiğimiz gibi, sayıyı doğrudan saklamak yerine **üç parçaya ayırıp** saklarız — işaret, normalize edilmiş anlamlı kısım ve üs. Aynı sayıda bit ile, üs sayesinde nokta gerektiğinde çok sola kayarak çok küçük sayıları, gerektiğinde çok sağa kayarak çok büyük sayıları temsil edebilir. Noktanın konumu artık sabit değildir; üs sayesinde ”yüzer”. Gösterimin adı da buradan gelir.

Kayan Noktalı Sayının Genel Formu:

$$(-1)^S \times 1.M \times 2^{E-bias}$$

Burada S işaret biti (0 pozitif, 1 negatif), M mantissa (anlamlı kısmın noktadan sonraki basamakları), E saklanan biaslı üs, ve $bias$ bir önceki paragrafta tanımladığımız sabit kaydırma değeridir.

1.6.2 Normalizasyon: Bedava Bir Bit

Bilgi Notu 1.16: Örtük Baştaki 1

İkilide (sıfır hariç) her sayı, tek bir şekilde $1.xxxx \times 2^k$ biçiminde normalize edilebilir; çünkü ikili sistemde ilk anlamlı basamak için tek bir seçenek vardır: ondalıkta ilk basamak 1’den 9’a kadar herhangi bir değer olabilirken, ikilide 0’dan farklı tek rakam 1’dir. Bu evrensel gerçek sayesinde, IEEE 754 formatında baştaki bu 1 hiçbir zaman saklanmaz. M (mantissa) alanına yalnızca 1’den sonraki basamaklar yazılır. Sonuçta, tek duyarlılık fiilen 24 bitlik bir hassasiyete sahip oluruz ama yalnızca 23 bit harcarız: baştaki 1 ”örtük” (implicit) olduğu için bedavadan kazanılmış bir bit budur.

Örnek 1.50: İki Normalizasyon Örneği: Büyük ve Küçük Bir Sayı

Büyük bir sayı: $(1101.011)_2$ ’yi normalize edelim. Nokta 3 basamak sola kaydırılır (ilk 1’in hemen sağına):

$$(1101.011)_2 = 1.101011 \times 2^3$$

Kontrol: $1.101011 \times 8 = 1101.011$, doğru.

Küçük bir sayı: $(0.000101)_2$ ’yi normalize edelim. Nokta 4 basamak sağa kaydırılır (ilk 1’in hemen sağına gelene kadar):

$$(0.000101)_2 = 1.01 \times 2^{-4}$$

Kontrol: $1.01 \times 2^{-4} = 1.25 \times 0.0625 = 0.078125$, ve $(0.000101)_2 = 2^{-4} + 2^{-6} = 0.0625 +$

$0.015625 = 0.078125$, eşleşiyor.

İki örnek birlikte gösteriyor ki: sayı ne kadar büyükse üs o kadar pozitif, sayı ne kadar küçükse (sıfıra o kadar yakınsa) üs o kadar negatif olur. Kayan nokta, üssü ayarlayarak her iki uca da uzanabilir.

1.6.3 IEEE 754 Formatları ve Bias

Tablo 1.3: IEEE 754 Tek ve Çift Duyarlıklılı Formatlar

Format	İşaret	Üs	Mantissa
Tek duyarlılık (32 bit)	1 bit	8 bit (bias = 127)	23 bit
Çift duyarlılık (64 bit)	1 bit	11 bit (bias = 1023)	52 bit

Bilgi Notu 1.17: Neden İşaretli Değil, Biaslı Üs?

Üs için 2 'nin tümleyeni yerine biaslı gösterim kullanılmasının mühendislik gerekçesi şudur: bu sayede iki kayan noktalı sayı, **sıradan işaretsiz tam sayı karşılaştırmacısıyla** karşılaştırılabilir. Biaslı gösterimde büyüklük sıralaması korunur. 2 'nin tümleyeni kullanılsaydı, negatif üsler (en soldaki bit 1) pozitif üslerden büyükmüş gibi görünürdü ve karşılaştırma devresi ek mantık gerektirirdi. Bias değeri, n bitlik üs alanı için $2^{n-1} - 1$ formülüyle seçilir; tek duyarlılıkta bu $2^7 - 1 = 127$ değerini verir.

1.6.4 Ondalıktan IEEE 754'e Dönüşüm

Örnek 1.51: $(0.6875)_{10}$ 'un Tek Duyarlıklılı IEEE 754 Gösterimi

Bu sayının ikili karşılığını zaten biliyoruz (Bölüm 1.2.9): $(0.6875)_{10} = (0.1011)_2$.

Normalize et: nokta bir basamak sağa kaydırılır, üssü telafi etmek için 2^{-1} çarpanı eklenir:

$$0.1011 = 1.011 \times 2^{-1}$$

İşaret: sayı pozitif olduğu için işaret biti $S = 0$ olarak belirlenir.

Üs: gerçek üs -1 'dir; biaslı üs $-1 + 127 = 126$ olarak hesaplanır, bu değer 8 bitte $(01111110)_2$ şeklinde yazılır.

Mantissa: normalize edilmiş formdaki baştaki 1 'den sonraki basamaklar (011) alınır, 23 bite tamamlanana kadar sıfırla doldurulur: 01100000000000000000000 .

Üç alan yan yana yazıldığında 32 bitlik sonuç elde edilir:

$$\underbrace{0}_{\text{S}} \underbrace{01111110}_{\text{E}} \underbrace{011000000000000000000000}_{\text{M}}$$

Alıştırma 1.12 ★★ $(2.5)_{10}$ 'un tek duyarlıklılı IEEE 754 gösterimini bulunuz.

Çözüm: Tam kısım $2 = (10)_2$, kesirli kısım $0.5 = (0.1)_2$ olduğundan $2.5 = (10.1)_2$ olarak yazılır. Normalize edilince $10.1 = 1.01 \times 2^1$ elde edilir. İşaret biti $S = 0$ 'dır (sayı pozitif). Gerçek üs 1 olduğundan biaslı üs $1 + 127 = 128 = (10000000)_2$ olarak bulunur. Mantissa, "01" basamaklarının

21 sıfırla tamamlanmasıyla 010000000000000000000000 olur. Üç alan birleştirilince:

0 10000000 010000000000000000000000

sonucuna ulaşılır.

Alıştırma 1.13 ★★★ $(-1.5)_{10}$ 'un tek duyarlıklı IEEE 754 gösterimini bulunuz.

Çözüm: Büyüklüğü $1.5 = (1.1)_2$ 'dir ve zaten normalize edilmiş durumdadır (1.1×2^0). Sayı negatif olduğundan işaret biti $S = 1$ olarak belirlenir. Gerçek üs 0 olduğundan biaslı üs $0 + 127 = 127 = (01111111)_2$ olarak bulunur. Mantissa, "1" basamağının 22 sıfırla tamamlanmasıyla 10000000000000000000000 olur. Üç alan birleştirilince:

1 01111111 10000000000000000000000000000000

sonucuna ulaşılır.

1.6.5 IEEE 754'ten Ondalığa Dönüşüm

Örnek 1.52: Bit Dizisinin Çözülmesi

1 10000001 010000000000000000000000 dizisini çözelim.

İşaret: $S = 1$ olduğundan sayı negatiftir.

Üs: biash üs $(10000001)_2 = 129$ 'dur; buradan gerçek üs $129 - 127 = 2$ olarak bulunur.

Mantissa: $M = 01 \dots 0$ 'dır; örtük baştaki 1 ile birlikte anlamlı kısım 1.01 olarak okunur.

Bu üç alan birleştirildiğinde değer şu şekilde hesaplanır:

$$(-1)^1 \times 1.01_2 \times 2^2 = -(101)_2 = -5$$

Alıştırma 1.14 ★★ $0\ 10000010\ 110000000000000000000000$ dizisini çözümleyip ondalık değerini bulunuz.

Çözüm: İşaret biti $S = 0$ olduğundan sayı pozitifdir. Biasl üs $(10000010)_2 = 130$ 'dur; buradan gerçek üs $130 - 127 = 3$ olarak bulunur. Mantissa $M = 11...0$ olduğundan, örtük baştaki 1 ile birlikte anlamlı kısım 1.11 olarak okunur. Bu üç alan birleştirildiğinde değer şu şekilde hesaplanır:

$$(-1)^0 \times 1.11_2 \times 2^3 = (1110)_2 = 14$$

1.6.6 Özel Değerler ve Kapanış

Özel Bit Kalıpları: Üs alanının tamamen 0 veya tamamen 1 olması özel anlamlar taşır; her durum için tek duyarlıklılı (32 bit) bir örnek aşağıda verilmiştir.

- **Üs tümü 0, mantissa tümü 0:** ± 0 değerini temsil eder, işaret biti hangisi olduğunu belirler (ör. 0 00000000 0 ... 0 ifadesi $+0$ 'ı, 1 00000000 0 ... 0 ifadesi -0 'ı verir). İlginçtir: 2'nin tümleyeninde tek bir sıfır kodu elde etmek için özenle tasarım yapmıştık (Bölüm 1.5.4), ama IEEE 754'te $+0$ ile -0 yine iki farklı bit kalıbı olarak karşımıza çıkar.
- **Üs tümü 1, mantissa tümü 0:** $\pm\infty$ değerini temsil eder (ör. 0 11111111 0 ... 0 ifadesi $+\infty$ 'u verir). Örneğin çok büyük iki sayının çarpımı, temsil edilebilir aralığı aştığında bu değer üretilir.
- **Üs tümü 1, mantissa sıfır değil:** NaN (Not a Number) değerini temsil eder (ör.

0 11111111 10...0), 0/0 ya da $\infty - \infty$ gibi tanımsız işlemlerin sonucudur.

- **Üs tümü 0, mantissa sıfır değil:** *denormalize* (subnormal) sayıları temsil eder (ör. 0 00000000 0...01). Bu durumda örtük baştaki 1 varsayımı bırakılır ve değer $(-1)^S \times 0.M \times 2^{1-bias}$ formülüyle hesaplanır. Amaç, sıfıra çok yakın sayıların aniden değil kademeli olarak küçülmesini sağlamaktır.

NaN kavramı soyut kalsın; IEEE 754'ün tanımladığı en tuhaf davranışlardan biri, üç yaygın programlama dilinde de birebir aynı şekilde karşımıza çıkar: **NaN, kendisine bile eşit değildir** ($x \neq x$). Bu, bir hata değil, standardın kendisinin bilinçli bir tasarım kararıdır — bu yüzden her dil, eşitlik operatörü yerine ayrı bir `isnan/isNaN` fonksiyonu sağlar.

C:

```
1 #include <stdio.h>
2 #include <math.h>
3
4 int main(void) {
5     double x = 0.0 / 0.0;           /* NaN üretir */
6     printf("x = %f\n", x);          /* "x = nan" */
7     printf("x == x: %d\n", x == x); /* 0 (yanlış!) */
8     printf("isnan(x): %d\n", isnan(x)); /* 1 (doğru yöntem) */
9     return 0;
10 }
```

Kod 1.1: C’de NaN üretimi ve $x \neq x$

C++:

```
1 #include <iostream>
2 #include <cmath>
3 #include <limits>
4
5 int main() {
6     double x = std::numeric_limits<double>::quiet_NaN();
7     std::cout << "x = " << x << "\n";
8     std::cout << "x == x: " << (x == x) << "\n"; // false
9     std::cout << "isnan(x): " << std::isnan(x) << "\n"; // true
10    return 0;
11 }
```

Kod 1.2: C++’ta NaN üretimi ve $x \neq x$

Java:

```
1 public class NanOrnegi {
2     public static void main(String[] args) {
3         double x = 0.0 / 0.0;           // NaN üretir
4         System.out.println("x = " + x); // "x = NaN"
5         System.out.println("x == x: " + (x == x)); // false
6         System.out.println("Double.isNaN(x): " + Double.isNaN(x)); // true
7     }
8 }
```

Kod 1.3: Java’da NaN üretimi ve $x \neq x$

Üçünde de aynı sonuç: $x == x$ karşılaştırması `false/0` döner, oysa her tür sağduyu bize bir sayının kendisine eşit olması gerektiğini söyler. Bunun nedeni, IEEE 754 standardının NaN

karşılaştırmalarını *tanımsız* kabul etmesi ve tüm karşılaştırma operatörlerinin ($==$, $<$, $>$ vb.) NaN içeren her işlemde *false* döndürmesini zorunlu kılmasıdır. $\neq$ operatörü bu kuralın tek istisnasıdır ve NaN için her zaman *true* döner. Bu yüzden bir değişkenin NaN olup olmadığını kontrol etmek için asla $x == x$ gibi bir eşitlik testi değil, dilin sağladığı özel fonksiyon (`isnan/std::isnan/Double.isNaN`) kullanılmalıdır.

Bilgi Notu 1.18: Artık Biliyoruz: $0.1 + 0.2 \neq 0.3$

Daha önce (Bölüm 1.2.9) $(0.1)_{10}$ 'un ikilide hiç sonlanmadığını görmüştük. IEEE 754'te mantissa **sonlu** (23 ya da 52 bit) olduğundan, sonlanmayan bu sayı en yakın temsil edilebilir değere **yuvarlanır** — gerçek 0.1'den son derece küçük ama sıfır olmayan bir farkla. 0.1 ve 0.2 ayrı ayrı yuvarlandıktan sonra toplanınca, bu küçük yuvarlama hataları birikir ve sonuç tam olarak 0.3'ün temsiline eşit çıkmaz. Bu, bir yazılım hatası değil, sonlu bit genişliğinin kaçınılmaz bir sonucudur.

1.7 İkili Kodlama Sistemleri

Şimdiye kadar Bölüm 1 boyunca hep *sayıları* ikiliye çevirmekle uğraştık. Ama bir bilgisayarın işlediği bilginin çoğu aslında sayısal bir büyüklük değildir: harfler, semboller, ya da "hangi çalışan", "hangi renk" gibi kategorik kimlikler de ikili düzende temsil edilmelidir. Bu bölümde, sayısal değeri değil bir *kimliği* temsil eden bit dizilerini, yani **kodları**, inceleyeceğiz.

İkili Kod: Bir kümede N farklı eleman varsa, bu elemanları ayırt eden bir n -bit *ikili kod*, her elemana 2^n olası bit kombinasyonundan benzersiz birini atar. Ayırt etmek için gereken minimum bit sayısı $n = \lceil \log_2 N \rceil$ 'dir, çünkü n bit tam olarak 2^n farklı kombinasyon üretir. Ancak **maksimum** bit sayısına dair bir sınır yoktur: N elemanı kodlamak için gerekenden fazla bit de kullanılabilir, fazladan kombinasyonlar basitçe kullanılmadan bırakılır.

Örneğin, 10 ondalık rakamı (0-9) ayırt etmek için teorik minimum $n = 4$ bittir, çünkü $2^3 = 8 < 10 \leq 2^4 = 16$. Bu durumda 16 olası kombinasyondan yalnızca 10 tanesi kullanılır; geri kalan 6 kombinasyon (1010'dan 1111'e kadar) tanımsız/kullanılmaz kalır. Birazdan göreceğimiz BCD kodu tam olarak bu stratejiyi izler.

1.7.1 BCD (Binary-Coded Decimal) Kodu

En yaygın kullanılan ondalık kod, her ondalık rakamı **ayrı ayrı** 4 bitlik ikili karşılığına çevirip bu grupları yan yana dizen **BCD** (Binary-Coded Decimal, ikili-kodlanmış ondalık) kodudur. BCD'ye bazen **8421 kodu** da denir, çünkü her bitin ağırlığı sırasıyla 8, 4, 2, 1'dir — yani her 4-bitlik grup, doğrudan o rakamın kendi ikili karşılığıdır.

BCD Kodu: k ondalık haneli bir sayının BCD karşılığı $4k$ bit gerektirir; her hane, 0000'dan 1001'e kadarki (yani 0'dan 9'a kadarki) 4-bit ikili karşılığıyla ayrı ayrı kodlanır. 1010'dan 1111'e kadarki 6 kombinasyon BCD'de **geçersizdir**, hiçbir anlamı yoktur.

Önemli: BCD, bir sayının ikili karşılığıyla **aynı şey değildir**. Bunu bir örnekle netleştirelim.

Örnek 1.53: $(185)_{10}$ 'in BCD ve İkili Karşılıklarının Karşılaştırılması

Gerçek ikili karşılık, sayının bütünü tek seferde ikiliye çevirerek bulunur:

$$(185)_{10} = (10111001)_2 \quad (8 \text{ bit yeterli})$$

BCD karşılık ise her haneyi **ayrı ayrı** kodlayıp yan yana dizerek bulunur:

$$1 \rightarrow 0001, \quad 8 \rightarrow 1000, \quad 5 \rightarrow 0101 \quad \Rightarrow \quad (185)_{10} = (0001 \ 1000 \ 0101)_{BCD}$$

BCD karşılık 12 bit gerektirirken gerçek ikili karşılık yalnızca 8 bit yeterlidir — aynı sayı için iki tamamen farklı bit deseni.

BCD, gerçek ikiliden daha fazla bit harcadığı halde neden kullanılır? Çünkü BCD bir **hız ya da verimlilik** kodu değil, bir **insan-makine arayüz kolaylığı** kodudur. Bilgisayarın girdi/çıkırtısını üreten insan ondalık sistemle düşünür; bir ekrana "185" yazdırmak ya da klavyeden "1", "8", "5" tuşlarını okumak, sayının tamamını ikiliye çevirip geri çevirmektense, her hanenin kendi 4-bit kutusunda ayrı ayrı durduğu bir temsille çok daha doğrudandır.

Alıştırma 1.15 ★ $(84)_{10}$ sayısının BCD karşılığını bulunuz.

Çözüm: $8 \rightarrow 1000, 4 \rightarrow 0100$, yan yana dizilince $(84)_{10} = (1000 \ 0100)_{BCD}$.

1.7.2 BCD Toplama

BCD'nin en can sıkıcı yönü burada ortaya çıkar: düz ikili toplama kuralları BCD'de doğrudan işlemez. İki BCD hanesi (ve olası bir önceki elde) toplandığında, sonuç en fazla $9 + 9 + 1 = 19$ olabilir; bu iki 4-bit grubu sıradan ikili sayıların gibi toplarsak, donanım bize doğru *ikili* toplamı verir (0'dan 19'a kadar). Ama BCD'de yalnızca 0000-1001 (0-9) geçerlidir. Toplam 1010 (10) veya üzerindeyse, ya da 4. bitten bir elde taşıyorsa, elde edilen sonuç BCD açısından geçersizdir.

BCD Toplama Kuralı (+6 Düzeltmesi): İki BCD hanesi (ve varsa önceki elde) sıradan ikili sayıların gibi toplanır. Elde edilen 4-bitlik ikili toplam:

- ≤ 1001 (9) ise ve 4. bitten bir elde taşmadıysa, sonuç zaten geçerli bir BCD hanesidir, düzeltme gerekmez.
- ≥ 1010 (10) ise **veya** 4. bitten bir elde taşıyorsa, kalan 4 bite 0110 (6) eklenir; bu hem doğru BCD hanesini üretir hem de (gerekmiyorsa) bir sonraki haneye giden elde bitini doğurur.

Düzeltilmenin neden tam 6 olduğu, 4 bitin sağladığı 16 kombinasyondan yalnızca 10'unun BCD'de geçerli olmasından kaynaklanır: $16 - 10 = 6$. Toplam geçersiz bölgeye (10-15) girdiğinde, 6 eklemek onu bu altı kullanılmayan kodun üzerinden atlatarak doğru hane ve elde bitini üretir.

Örnek 1.54: Düzeltme Gerekmeyen Toplam: $4 + 5$

$$\begin{array}{r} 0100 \\ + 0101 \\ \hline 1001 \end{array}$$

İkili toplam $1001 (9) \leq 1001$ olduğundan zaten geçerli bir BCD hanesidir, düzeltme gerekmez.

Örnek 1.55: Düzeltme Gereken Toplam: $4 + 8$

$$\begin{array}{r} 0100 \\ + 1000 \\ \hline 1100 \end{array}$$

İkili toplam $1100 (12) \geq 1010$ olduğundan geçersizdir; $0110 (6)$ eklenir:

$$\begin{array}{r} 1100 \\ + 0110 \\ \hline 1\ 0010 \end{array}$$

Sonuç: hane $0010 (2)$, elde 1 — yani "12" (elde = 1, hane = 2). Doğru!

Örnek 1.56: Taşan Elde ile Düzeltme: $8 + 9$

$$\begin{array}{r} 1000 \\ + 1001 \\ \hline 1\ 0001 \end{array}$$

Burada zaten 4. bitten bir elde taşmış durumdadır. Kalan 4 bite yine 0110 eklenir:

$$\begin{array}{r} 0001 \\ + 0110 \\ \hline 0111 \end{array}$$

Elde zaten mevcuttu, kalan hane $0111 (7)$ — sonuç "17" (elde = 1, hane = 7). Doğru!

Bu tek haneli kural, çok haneli sayılara şöyle genişler: hanelere sağdan sola, birer birer uygulanır ve her hanenin ürettiği elde bir sonraki (daha soldaki) hanenin toplamına dahil edilir — tıpkı okulda öğrendiğimiz elle toplamada olduğu gibi.

Örnek 1.57: Çok Haneli BCD Toplama: $(184)_{BCD} + (576)_{BCD}$

Sağdan sola, hane hane:

Hane	İkili Toplam (elde dahil)	Düzeltme	Sonuç / Elde
Birler	$0100 + 0110 = 1010$	≥ 1010 , +0110	$0000 / 1$
Onlar	$1000 + 0111 + 0001 = 1\ 0000$	elde taşı, +0110	$0110 / 1$
Yüzler	$0001 + 0101 + 0001 = 0111$	≤ 1001 , gerekmez	$0111 / 0$

Elde zincirinin izlenmesi önemlidir: birler hanesinin ürettiği elde (1), onlar hanesinin toplamına üçüncü terim olarak girmiştir ($1000 + 0111 + 0001$); onlar hanesinin ürettiği elde (1) de aynı şekilde yüzler hanesinin toplamına girmiştir ($0001 + 0101 + 0001$).

Yüzler hanesinde üretilen elde 0 olduğundan zincir burada sona erer — eğer 1 olsaydı, dördüncü bir (binler) haneye taşınacaktı. Sağdan sola haneler birleştirilince: $(0111\ 0110\ 0000)_{BCD} = (760)_{10}$. Doğrulama: $184 + 576 = 760$. ✓

Alıştırma 1.16 ★ $4 + 6$ toplamının BCD karşılığını (gerekliyorsa düzeltme dahil) bulunuz.

Çözüm: $0100 + 0110 = 1010$ (10), ≥ 1010 olduğundan 0110 eklenir: $1010 + 0110 = 1\ 0000$. Sonuç: hane 0000 (0), elde 1 — yani BCD toplamı $(1\ 0000)_{BCD}$, ondalıkta 10.

1.7.3 Diğer Ondalık Kodlar

BCD, dört bitle on rakamı kodlamanın **tek** yolu değildir — sadece en yaygın olanıdır. Aynı 4 bittten 10 farklı kombinasyon seçmenin başka yolları da vardır, ve her seçim farklı bir mühendislik avantajı sağlar.

Ağırlıklı Kod: BCD’de her bitin sabit bir *ağırlığı* vardır (8, 4, 2, 1) ve bir hanenin değeri, 1 olan bitlerin ağırlıklarını toplayarak bulunur. Aynı fikir farklı ağırlık kümeleriyle de uygulanabilir; buna **ağırlıklı kod** denir. Ağırlıkların hepsinin pozitif olması da gerekmez.

Örnek 1.58: 2421 Kodu: Aynı Rakam İçin İki Geçerli Kodlama

2421 kodunda ağırlıklar 2, 4, 2, 1’dir. 0100 bit kalıbını değerlendirelim: $2 \cdot 0 + 4 \cdot 1 + 2 \cdot 0 + 1 \cdot 0 = 4$. Şimdi 1010’u değerlendirelim: $2 \cdot 1 + 4 \cdot 0 + 2 \cdot 1 + 1 \cdot 0 = 4$. **Aynı değer, iki farklı bit kalıbı!** 2421 kodunda bazı rakamlar için birden fazla geçerli kodlama vardır; bu seçim özgürlüğü rastgele değildir — birazdan göreceğimiz *öz-tümleyen* özelliğini sağlamak için kasıtlı olarak kullanılır.

Ağırlıklı olmayan bir kod da mümkündür: **Excess-3** (fazla-3) kodunda hiçbir ağırlık yorumu yoktur, kural yalnızca şudur: *her rakamın düz ikili karşılığına 3 ekleyin*. Örneğin rakam 5: $0101 + 0011 = 1000$.

Aşağıdaki tablo, dört farklı ondalık kodu karşılaştırmalı olarak gösterir.

Rakam	BCD (8421)	2421	Excess-3	8, 4, -2, -1
0	0000	0000	0011	0000
1	0001	0001	0100	0111
2	0010	0010	0101	0110
3	0011	0011	0110	0101
4	0100	0100	0111	0100
5	0101	1011	1000	1011
6	0110	1100	1001	1010
7	0111	1101	1010	1001
8	1000	1110	1011	1000
9	1001	1111	1100	1111

Bu özelliğin neye yaradığını anlamak için önce nereden geldiğini hatırlayalım: Bölüm 1.4’de çıkarmayı toplamaya çevirmek için tümleyen almıştık; aynı numarayı bir sonraki alt bölümde

ondalık sayılar için de yapacağız, ve bu her seferinde bir rakamın 9'un tümleyenini ($9 - d$) hesaplamayı gerektirecek. Bunu donanımda hesaplamamızın iki yolu var:

- **Aritmetik yoldan:** Girdisi d , çıktısı $9 - d$ olan gerçek bir çıkarma devresi (subtractor) kurulur. Bu yol, kapı sayısı bakımından pahalı ve gecikme bakımından yavaştır.
- **Kod seçimiyle:** Rakamlar öyle bir kodla saklanırsa ki, kodun bitlerini teker teker **tersine çevirmek** ($1 \rightarrow 0, 0 \rightarrow 1$) otomatik olarak $9 - d$ 'nin kodunu verir; bu durumda çıkarma devresine hiç gerek kalmaz, yalnızca 4 paralel NOT kapısı yeterli olur.

Burada terminolojiyi netleştirmek gerekir: bir **kodlama şeması** (Excess-3, BCD gibi), her rakama bir **bit kalıbı** atayan kuraldır. Aşağıdaki tanım bu iki kavramı ayrı tutarak ifade edilmiştir.

Öz-Tümleyen (Self-Complementing) Kod: Bir kodlama şemasında her rakama bir bit kalıbı atanır. Bu şema, herhangi bir rakamın bit kalıbındaki tüm bitler tersine çevrildiğinde ($1 \rightarrow 0, 0 \rightarrow 1$) ortaya çıkan yeni bit kalıbı, tam olarak $9 - d$ rakamına atanmış bit kalıbıyla aynı oluyorsa **öz-tümleyen** olarak adlandırılır.

Bunun gerçekten çalıştığını Excess-3 üzerinden, adım adım görelim. Ama önce, bu kodun tanımında ilk bakışta kafa karıştırabilecek bir noktayı vurgulayalım: Excess-3 kuralı, "*rakam d 'nin bit kalıbı, $d + 3$ sayısının düz ikili karşılığıdır*" der. Yani bir bit kalıbının **sayısal değeri** ile o kalıbın **temsil ettiği rakam**, aralarında sabit bir $+3$ fark olacak şekilde, birbirinden farklıdır. Kodun adındaki "excess" ("fazlalık") kelimesi tam olarak bu farkı anlatır.

Örnek 1.59: Excess-3'ün Öz-Tümleyen Özelliği

Adım 1 – rakam 3'ün bit kalıbını bulalım: Rakam 3'ün düz ikili karşılığı 0011'dir. Excess-3 kuralı gereği buna 3 eklenir (yani $0011 + 0011$):

$$0011 + 0011 = 0110$$

Burada dikkat edilmesi gereken can alıcı nokta şudur: 0110'nın **sayısal değeri** 6'dır ($4 + 2 = 6$). Ama bu bit kalıbı, rakam 6'yı değil, **rakam 3'ü** temsil etmek üzere atanmıştır – çünkü kuralımız " $d + 3$ " idi ve $3 + 3 = 6$. Yani rakam 3'ün bit kalıbı, sayısal değeri 6 olan 0110'dır; bu bir çelişki değil, kodun tanımının doğal bir sonucudur.

Adım 2 – aynı kuralı rakam 6'ya uygulayalım: Rakam 6'nın düz ikili karşılığı 0110'dır. Buna 3 eklenir: $0110 + 0011 = 1001$. 1001'in sayısal değeri 9'dur, ama bu bit kalıbı **rakam 6'yı** temsil eder (yine $+3$ farkla: $6 + 3 = 9$).

Adım 3 – şimdi hiç toplama yapmadan, sadece Adım 1'in bit kalıbının bitlerini çevirelim: Rakam 3'ün bit kalıbı 0110'ı ele alalım, her bitini tek tek tersine çevirelim:

$$\begin{array}{ccccccc} \underline{0} & \underline{1} & \underline{1} & \underline{0} & & \Rightarrow & 1001 \\ \rightarrow 1 & \rightarrow 0 & \rightarrow 0 & \rightarrow 1 & & & \end{array}$$

Adım 4 – karşılaştırma: Adım 2'de toplama yaparak bulduğumuz rakam 6'nın bit kalıbı (1001), Adım 3'te hiç toplama yapmadan, sadece rakam 3'ün bit kalıbının bitlerini çevirerek bulduğumuz sonuçla (1001) birebir aynıdır. Yani: rakam 3'ün bit kalıbının bitlerini çevirmek, bize doğrudan rakam $9 - 3 = 6$ 'nın bit kalıbını verdi – hiçbir toplama/çıkarma yapmadan.

Bu bir rastlantı değildir; "3" sayısının seçimi de rastgele değildir — kanıtlayalım. Bir 4-bitlik sayının bitlerini tek tek çevirmek, matematiksel olarak $15 - x$ işlemine eşittir (çünkü 4 bit 0'dan 15'e kadar sayar, ve her biti çevirmek sayıyı 15'e göre "aynalar": $0000 \rightarrow 1111$ yani $0 \rightarrow 15$, $0001 \rightarrow 1110$ yani $1 \rightarrow 14$, ve genel olarak $x \rightarrow 15 - x$). Rakam d 'nin bit kalıbının sayısal değeri $d + 3$ olduğundan, bu bit kalıbının bitlerini çevirmek bize $15 - (d + 3) = 12 - d$ değerinde bir bit kalıbı verir. Öte yandan, bizim aslında elde etmek istediğimiz şey, rakam $9 - d$ 'ye atanmış bit kalıbıdır; bu kalıbın sayısal değeri de $(9 - d) + 3 = 12 - d$ 'dir. **İkisi de $12 - d$ — birebir eşit!** Bu sabit ekleme miktarına (Excess-3'te 3, BCD'de 0) kodun **ofseti** denir. Ofset, bir bit kalıbının sayısal değeri ile o kalıbın temsil ettiği rakam arasındaki sabit farktır. İşte Excess-3'ün ofseti olan "3", tam olarak yukarıdaki eşitliği ($15 - 3 = 9 + 3$) sağlayacak şekilde seçilmiştir; rastgele bir tercih değildir.

Bu özellik neden önemlidir? Çünkü ondalık çıkarmada (bir sonraki alt bölümde göreceğimiz gibi) bir tümleyen alma işlemi normalde ayrı bir çıkarma devresi gerektirir — ama öz-tümleyen bir kodda bu işlem tek bir NOT kapısı katmanına indirgenir, ekstra donanımına hiç gerek kalmaz.

Alıştırma 1.17 ★ BCD (8421) kodunun öz-tümleyen olmadığını, yukarıdaki kanıtla aynı yöntemi kullanarak gösteriniz.

Çözüm: BCD'nin ofseti 0'dır, yani bit kalıbının sayısal değeri rakamın kendisine eşittir ($d + 0 = d$). Excess-3'teki "+3" yerine burada "+0" vardır. Bit çevirme yine $15 - x$ işlemine eşittir; BCD kodunu çevirirsek $15 - d$ elde ederiz. Ama bizim istediğimiz, $9 - d$ 'nin BCD kodu, yani sadece $9 - d$ 'nin kendisi (ofset 0 olduğu için). Öz-tümleyen olması için $15 - d = 9 - d$ eşitliğinin sağlanması gerekirdi — bu ise d 'den bağımsız olarak asla doğru olamaz ($15 \neq 9$). Somut bir örnekle doğrulayalım: 0'ın BCD kodu 0000'dır; bitleri çevirirsek 1111 (15) elde ederiz. Ama $9 - 0 = 9$ 'un BCD kodu 1001'dir; $1111 \neq 1001$. Demek ki BCD'nin öz-tümleyen olmaması tek bir rakama özgü bir tesadüf değil, BCD'nin ofsetinin (0) Excess-3'ün gerektirdiği değere (3) eşit olmamasından kaynaklanan **yapısal bir imkansızlıktır**.

Son olarak, yukarıdaki tablonun son sütunu bir **negatif ağırlıklı kod** örneğidir: 8, 4, -2, -1 kodunda ağırlıkların hepsi pozitif olmak zorunda değildir. Örneğin 0110 bu kodda $8 \cdot 0 + 4 \cdot 1 + (-2) \cdot 1 + (-1) \cdot 0 = 2$ olarak yorumlanır — ve bu kod da (2421 gibi) öz-tümleyendir.

1.7.4 BCD'de İşaretli Ondalık Aritmetik (10'un Tümleyenini)

Bölüm 1.5'de ikili sayılarda negatifleri saklamak için 2'nin tümleyenini kullanmış, çıkarmayı toplamaya çevirmiştik. Aynı fikri şimdi BCD ile saklanan ondalık sayılara uygulayacağız; bu sefer kullanılan araç 10'un tümleyenidir.

İşaret Hanesi: Bir işaretli BCD sayısında işaret, ayrı bir bit değil, ayrı bir **hane** olarak saklanır: bu hane pozitif sayılar için 0000, negatif sayılar için 1001 (rakam 9'un BCD kodu) değerini alır.

Neden özellikle 9 seçilmiştir? Çünkü 10'un tümleyenini alma işlemi, negatif bir sayının en soluna hep 9 hanesi(leri) dizer. Bu, 2'nin tümleyeninde negatif sayıların en soluna 1 bitleri dizilmesinin (işaret genişletme) ondalıktaki karşılığıdır.

Bir BCD Sayısının 10'un Tümleyenini Bulma: İki adımlı bir işlemdir:

1. Her hanenin 9'un tümleyeni alınır (her rakam d için $9 - d$ hesaplanır); buna 9'un tümleyeni denir.
2. Sonucun en sağdaki hanesine 1 eklenir.

Bilgi Notu 1.19: Öz-Tümleyen Kodlarla Bağlantı

Bir önceki alt bölümde BCD'nin öz-tümleyen olmadığını kanıtlamıştık. Bunun donanımsal bedeli burada ortaya çıkar: yukarıdaki 1. adımda her hanenin 9'un tümleyenini bulmak gerçek bir çıkarma gerektirir. Rakamlar öz-tümleyen bir kodla (Excess-3 gibi) saklansaydı, bu adım yalnızca bit çevirmeyle (NOT kapılarıyla) yapılabilirdi.

İşaretili Ondalık Toplama Kuralı: İşaret hanesi dahil tüm haneler, BCD toplama kuralıyla (gerektiğinde +6 düzeltmesiyle) toplanır. En soldaki haneden (işaret hanesinden) taşan son elde atılır, tıpkı 2'nin tümleyeninde işaret bitinden taşan eldenin atılması gibi.

Örnek 1.60: $(+375) + (-240)$

Önce -240 'ı 10'un tümleyeniyle temsil edelim. 240 'ın 9'un tümleyeni: $9 - 2 = 7$, $9 - 4 = 5$, $9 - 0 = 9$, yani 759. En sağdaki haneye 1 eklenir: 760. İşaret hanesi 9 olduğundan, $-240 \rightarrow 9760$.

Şimdi $0375 + 9760$ toplamını hane hane (sağdan sola) yapalım. Hatırlatma: bir hanenin toplamı 9'u aşarsa (ya da elde üretirse), Bölüm 1.7.2'da gördüğümüz +6 BCD düzeltmesi uygulanır:

Hane	Toplam	Düzeltilme (+6 BCD kuralı)	Sonuç / Elde
Birler	$5 + 0 = 5$	gerekmez	5 / 0
Onlar	$7 + 6 = 13$	≥ 10 : +6 eklenir	3 / 1
Yüzler	$3 + 7 + 1 = 11$	≥ 10 : +6 eklenir	1 / 1
İşaret	$0 + 9 + 1 = 10$	≥ 10 : +6 eklenir	0 / 1 (atılır)

Son elde (işaret hanesinden taşan) atılır. Sonuç haneleri (işaret, yüzler, onlar, birler) 0, 1, 3, 5'tir, yani +135. Doğrulama: $375 - 240 = 135$.

Alıştırma 1.18 ★★ İki parçada çözünüz: (a) $370 + (-250)$, (b) $250 + (-370)$. İkinci kısımda sonucun işaret hanesi negatif çıkacaktır; gerçek büyüklüğü bulmak için sonucun büyüklük hanelerinin 10'un tümleyenini alın.

Çözüm (a): -250 'nin 10'un tümleyeni: 9'un tümleyeni $9 - 2 = 7$, $9 - 5 = 4$, $9 - 0 = 9$, yani 749; en sağa 1 eklenir: 750; işaret hanesi 9: $-250 \rightarrow 9750$.

Hane	Toplam	Düzeltilme (+6 BCD kuralı)	Sonuç / Elde
Birler	$0 + 0 = 0$	gerekmez	0 / 0
Onlar	$7 + 5 = 12$	≥ 10 : +6 eklenir	2 / 1
Yüzler	$3 + 7 + 1 = 11$	≥ 10 : +6 eklenir	1 / 1
İşaret	$0 + 9 + 1 = 10$	≥ 10 : +6 eklenir	0 / 1 (atılır)

Sonuç haneleri 0, 1, 2, 0'dır, yani +120. Doğrulama: $370 - 250 = 120$.

Çözüm (b): -370 'in 10 'un tümleyeni: 9 'un tümleyeni $9 - 3 = 6$, $9 - 7 = 2$, $9 - 0 = 9$, yani 629; en sağa 1 eklenir: 630; işaret hanesi 9: $-370 \rightarrow 9630$.

Hane	Toplam	Düzeltilme (+6 BCD kuralı)	Sonuç / Elde
Birler	$0 + 0 = 0$	gerekmez	0 / 0
Onlar	$5 + 3 = 8$	gerekmez	8 / 0
Yüzler	$2 + 6 = 8$	gerekmez	8 / 0
İşaret	$0 + 9 = 9$	gerekmez	9 / 0

Sonuç haneleri 9, 8, 8, 0'dır. İşaret hanesi 9 olduğundan sonuç negatiftir; gerçek büyüklüğü bulmak için büyüklük hanelerinin (880) 10 'un tümleyenini alalım: 9 'un tümleyeni $9 - 8 = 1$, $9 - 8 = 1$, $9 - 0 = 9$, yani 119; en sağa 1 eklenir: 120. Sonuç -120 'dir. Doğrulama: $250 - 370 = -120$.

Ondalık çıkarma ($M - N$) da ikilideki gibi yapılır: N 'nin (çıkanın) 10 'un tümleyeni alınıp M 'ye eklenir.

1.7.5 Gray Kodu

Düz ikili sayma sırasında, ardışık iki sayı arasında **birden fazla bit aynı anda** değişebilir. Örneğin $7 \rightarrow 8$ geçişi $0111 \rightarrow 1000$ 'dir: dört bitin dördü de aynı anda değişir.

Bu, kağıt üzerinde bir sorun değildir. Gerçek donanımda ise ciddi bir soruna yol açabilir: farklı bitlerin mantık kapılarından geçip yeni değerlerine ulaşması tam olarak aynı anda gerçekleşmez. Üretim toleransları, kapı gecikmeleri ve tel uzunlukları gibi nedenlerle, bitler arasında nanosaniyeler mertebesinde gecikme farkları oluşur. $0111 \rightarrow 1000$ geçişinde, eğer örneğin en sağdaki bit diğerlerinden biraz daha yavaş değişirse, devre kısa bir süreliğine 1001 gibi hiç var olmaması gereken, yanlış bir ara değer üretebilir.

Gray Kodu: Gray kodu, ardışık iki değer arasında **yalnızca tek bir bitin** değiştiği bir ikili kodlama şemasıdır.

Aşağıdaki tablo, 0'dan 15'e kadar düz ikili ile Gray kodunu karşılaştırmalı gösterir. Her satırdan bir sonrakine geçerken Gray sütununda yalnızca tek bir bitin değiştiğine dikkat edin.

Ondalık	İkili	Gray Kodu
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Örnek 1.61: 7 → 8 Geçişinde İkili ile Gray Kodunun Karşılaştırılması

Düz ikilide 7 → 8 geçişi 0111 → 1000'dir: dört bitin dördü de değişir. Gray kodunda aynı geçiş 0100 → 1100'dür: yalnızca en soldaki bit değişir, diğer üç bit aynı kalır. Geçiş sırasında hangi an yakalanırsa yakalansın, okunan değer ya eski değer ya da yeni değerdir. İki değer karışımı olan geçersiz bir ara değer hiçbir zaman ortaya çıkmaz.

Gray kodunun tipik bir kullanım alanı, dönen bir milin açısal konumunu ölçen sensörlerdir (encoder). Mil döndükçe, üzerindeki kodlanmış disk bir sensör tarafından okunur. Düz ikili kullanılsaydı, birden fazla bitin aynı anda değiştiği geçiş anlarında yanlış okuma riski olurdu; Gray kodu bu riski ortadan kaldırır.

Bilgi Notu 1.20: İkiliden Gray Koduna Dönüşüm

Yukarıdaki tabloyu ezbere bilmeye gerek yoktur; bir ikili sayıdan Gray koduna doğrudan bir formülle geçilebilir. n bitlik ikili sayı $b_{n-1} \dots b_1 b_0$ için, Gray kodunun bitleri $g_{n-1} \dots g_1 g_0$ şöyle hesaplanır:

$$g_{n-1} = b_{n-1}, \quad g_i = b_{i+1} \oplus b_i \quad (0 \leq i \leq n-2)$$

Burada $\oplus$ sembolü XOR (özel-veya) işlemidir; Bölüm 2'de tanıtılacaktır. Yani en soldaki bit aynen korunur, diğer her bit ise kendisinden bir solundaki bitle XOR'lanır.

Örnek 1.62: Formülle 7 = (0111)₂'nin Gray Koduna Çevrilmesi

$b_3 b_2 b_1 b_0 = 0111$ için:

$$g_3 = b_3 = 0, \quad g_2 = b_3 \oplus b_2 = 0 \oplus 1 = 1$$

$$g_1 = b_2 \oplus b_1 = 1 \oplus 1 = 0, \quad g_0 = b_1 \oplus b_0 = 1 \oplus 1 = 0$$

Sonuç: $(0100)_2$. Tabloyla birebir uyuşuyor.

Alıştırma 1.19 ★ Yukarıdaki formülü kullanarak $(1010)_2$ 'nin $(10$ 'un ikili karşılığı) Gray kodunu bulunuz.

Çözüm: $b_3b_2b_1b_0 = 1010$. $g_3 = b_3 = 1$. $g_2 = b_3 \oplus b_2 = 1 \oplus 0 = 1$. $g_1 = b_2 \oplus b_1 = 0 \oplus 1 = 1$. $g_0 = b_1 \oplus b_0 = 1 \oplus 0 = 1$. Sonuç: $(1111)_2$. Tabloda 10 satırına bakıldığında Gray kodu 1111'dir. Doğrulandı.

1.7.6 ASCII Karakter Kodu

Şu ana kadar gördüğümüz kodlar (BCD, Gray) hep *rakamları* temsil ediyordu. Ama bir bilgisayarın işlemesi gereken bilginin çoğu rakam değildir; harfler, noktalama işaretleri ve klavyedeki her tuş da ikili düzende temsil edilmelidir.

Kaç bite ihtiyacımız var? Bölüm 1.7'deki İkili Kod tanımını hatırlayalım: n bit, 2^n farklı eleman ayırt eder. Temsil etmemiz gereken kümeyi sayalım: 26 büyük harf, 26 küçük harf, 10 rakam, noktalama/özel karakterler (% , \$, @ gibi) ve birkaç görünmez kontrol karakteri (satır başı, sekme gibi). Bu toplam 128'e kadar çıkar. $2^6 = 64$ yetmez, $2^7 = 128$ yeterlidir; bu yüzden **ASCII 7 bit** kullanır.

ASCII (American Standard Code for Information Interchange): ASCII, 128 karakteri (harfler, rakamlar, noktalama işaretleri ve kontrol karakterleri) 7-bit bir kodla temsil eden standart karakter kodudur. Bit dizisi, bu kitap boyunca kullandığımız 0-indeksli düzene uygun olarak $b_6b_5b_4b_3b_2b_1b_0$ şeklinde adlandırılır; b_6 en soldaki (en anlamlı) bit, b_0 en sağdaki (en az anlamlı) bittir.

Bilgi Notu 1.21: Tarihi Bir Not: ASCII'nin Orijinal Bit Numaralandırması

ASCII standardının 1963 tarihli ilk hâli ve onu izleyen birçok ders kitabı, bitleri b_1 'den b_7 'ye kadar (b_0 hiç kullanılmadan) adlandırır. Bu, telgraf/teleprinter mühendisliğinden kalma eski bir numaralandırmadır. Modern dijital tasarımda (ve bu kitabın geri kalanında, örn. Gray kodunda $b_{n-1} \dots b_1b_0$) standart olan 0-indeksli düzen kullanılır; burada tutarlılık için bu düzeni tercih ediyoruz.

Standart ASCII tablosu, 7 biti ikiye bölerek okunur: en soldaki 3 bit ($b_6b_5b_4$) tablonun **sütununu**, kalan 4 bit ($b_3b_2b_1b_0$) tablonun **satırını** belirler. Bu bölünme rastgele değildir: sütun 000-001 tamamen kontrol karakterlerine (görünmez, yazdırılamaz), sütun 010-011 rakamlara ve temel noktalamaya, sütun 100-101 büyük harflere, sütun 110-111 küçük harflere ayrılmıştır. Yani sütun numarası, karakterin kategorisini anında belli eder.

$b_3b_2b_1b_0 \setminus b_6b_5b_4$	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	'	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

Örnek 1.63: Harf 'A'nın ASCII Kodunun Bulunması

Tabloda 'A' harfi, sütun 100 ve satır 0001'de yer alır. Yani $b_6b_5b_4 = 100$, $b_3b_2b_1b_0 = 0001$, birleştirince: $(1000001)_2$.

Tabloya dikkatlice bakılırsa ilginç bir örüntü görülür: 'A' = 1000001, 'a' = 1100001. Aralarındaki tek fark b_5 bitidir: 0 ise büyük harf, 1 ise küçük harf. Bu, klavye ve yazılım tasarımını kolaylaştıran bilinçli bir tercihtir; büyük/küçük harf dönüşümü donanımda tek bir bitin çevrilmesine indirgenir.

Bilgi Notu 1.22: Neden 8 Bit (Byte) ile Saklanır?

ASCII yalnızca 7 bit gerektirdiği halde, bilgisayarlar veriyi 8-bitlik gruplar (*byte*) halinde işler. Fazladan kalan 8. bit boşa gitmez; genellikle **parity** (hata sezme) için kullanılır. Bir sonraki alt bölümün konusu tam olarak budur.

Alıştırma 1.20 ★ Harf 'Z'nin ASCII kodunu, yukarıdaki tablodan sütun/satır konumunu bularak yazınız.

Çözüm: 'Z' harfi, tabloda sütun 101 ve satır 1010'da yer alır. Yani $b_6b_5b_4 = 101$, $b_3b_2b_1b_0 = 1010$, birleştirince: $(1011010)_2$.

1.7.7 Hata Sezme: Parity Biti

Bir önceki alt bölümün sonunda bıraktığımız yerden devam edelim: ASCII 7 bit ($b_6 \dots b_0$) kullanılır, ama bilgisayarlar veriyi 8-bitlik byte'lar hâlinde işler. Fazladan kalan 8. bit genellikle **parity** için kullanılır. Bu bit, tam olarak b_7 'dir: 7-bit ASCII ($b_6 \dots b_0$) artı 1 parity biti (b_7), hiç boşluk kalmadan tam bir byte oluşturur.

Bir veri, bir kablodan veya bir depolama ortamından geçerken, elektriksel gürültü nedeniyle bir bit yanlışlıkla $0 \rightarrow 1$ veya $1 \rightarrow 0$ dönebilir. Parity biti, bu tür tek bitlik hataları sezmek için kullanılan basit bir yöntemdir.

Parity Biti: Parity biti, bir mesaja eklenen ve mesajdaki toplam 1 sayısını istenen bir pariteye (çift ya da tek) tamamlayan fazladan bir bittir.

- **Çift parity (even parity):** parity biti, toplam 1 sayısı çift olacak şekilde seçilir.
- **Tek parity (odd parity):** parity biti, toplam 1 sayısı tek olacak şekilde seçilir.

Sistemde ikisinden biri seçilir ve tutarlı biçimde kullanılır; çift parity daha yaygındır.

Örnek 1.64: Harf 'A' İçin Parity Biti Hesaplanması

'A' = 1000001 (7 bit). İçindeki 1 sayısını sayalım: 2 tanedir (zaten çift).

- Çift parity isteniyorsa, parity biti 0 kalır (toplam zaten çift): 01000001.
- Tek parity isteniyorsa, toplamı tek yapmak için parity biti 1 olur: 11000001.

Alıcı tarafta kontrol basittir: alıcı, gelen 8 bitteki 1 sayısını sayar. Beklenen pariteye (çift/tek) uymuyorsa, en az bir bitin yolda bozulduğu anlaşılır; bu durumda yeniden gönderim istenir (NAK, *Negative Acknowledge*, yani "olumsuz onay/red bildirimi"). Uyuyorsa aktarım onaylanır (ACK, *Acknowledge*, yani "onay bildirimi").

Bilgi Notu 1.23: Parity'nin Kritik Sınırlaması: Çift Sayıda Hatayı Yakalayamaz

Eğer **tek sayıda** bit (1, 3, 5, 7 tanesi) bozulursa, toplam 1 sayısının paritesi değişir ve hata sezilir. Ama **çift sayıda** bit (2, 4, ...) aynı anda bozulursa, toplam 1 sayısındaki değişimler birbirini götürür; parite değişmez ve hata sessizce kaçır.

Somut bir örnekle görelim: 'A'nın çift parity ile gönderilen hâli 01000001'dir (içinde 2 tane 1 vardır). Aktarım sırasında bu iki 1 biti de 0'a dönerse, alıcı 00000000 alır. Bu byte'taki 1 sayısı 0'dır, yine çift; parity kontrolü "hata yok" der, oysa veri tamamen farklıdır.

Bunun temel nedeni, parity bitinin veri hakkında çok kaba bir özet tutmasıdır: yalnızca toplam 1 sayısının tek mi çift mi olduğunu bilir, hangi bitlerin değiştiğine dair hiçbir bilgi taşımaz. İki bit birden değiştiğinde, birinin etkisi diğerininkini götürür ve toplam parite üzerindeki net etki sıfırlanır.

Peki bu zayıflığa rağmen parity neden hâlâ yaygın kullanılır? Çünkü gerçek dünyadaki gürültü kaynaklı hatalarda, aynı anda tam olarak iki (ya da dört, altı, ...) bitin birden bozulması istatistiksel olarak çok daha nadirdir. En sık karşılaşılan hata türü tek bir bitin bozulmasıdır, ve parity tam olarak bunu, tek bir ekstra bitle, ucuza yakalar. Daha güçlü hata sezme/düzeltilme gerektiren uygulamalarda (örn. bellek denetimi, ağ protokolleri) checksum, CRC (döngüsel artıklık denetimi) veya Hamming kodu gibi daha gelişmiş yöntemler kullanılır; bunlar bu kitabın kapsamı dışındadır.

Alıştırma 1.21 ★ A = 0101100 (7 bit) için çift parity bitini bulunuz.

Çözüm: 0101100 içindeki 1 sayısı: 3 tanedir (tek). Çift parity istediğimiz için, toplamı çift yapacak parity biti gerekir: $3 + 1 = 4$ (çift), yani parity biti 1'dir.

Bu, Bölüm 1.7'da başladığımız §1.7'nin son alt bölümüdür: BCD'den Gray koduna, AS-CII'den parity'ye kadar, sayıların ve sembollerin ikili düzende nasıl kodlandığını ve bu kodlamalardaki hataların nasıl sezilebileceğini gördük.

Bool Cebri ve Mantık Kapıları

Öğrenme Hedefleri

Bu bölümü tamamladığınızda aşağıdakileri yapabileceksiniz:

- İkili değişken ve mantık fonksiyonu kavramlarını tanımlamak.
- Bool cebrinin Huntington aksiyomlarını ve temel teoremlerini uygulamak.
- De Morgan teoremiyle Bool ifadelerini sadeleştirmek.
- Yedi temel mantık kapısının (AND, OR, NOT, NAND, NOR, XOR, XNOR) doğruluk tablolarını ve Bool ifadelerini yazmak.
- NAND/NOR kapılarının fonksiyonel tamlığının ASIC tasarımındaki önemini açıklamak.

2.1 Giriş

Önceki bölümde gördüğümüz ikili sayılar, bir bilgisayarın belleğinde *durağan* veriyi temsil eder: bir sayı, bir karakter, bir renk kodu. Ama bir işlemci sadece veri saklamaz — toplama yapar, karşılaştırır, dallanma kararı verir. Yani veriyi *işleyen* bir mekanizmaya ihtiyacımız var; bu mekanizma da yine 0 ve 1'lerle, ama bu kez statik değil, girişe bağlı olarak çıkış üreten devrelerle kurulur. İşte bu devrelerin arkasındaki matematik, konumuz olan *Bool cebridir*.

Sayısal donanım tasarımının bu ikinci temel taşı, 1854'te George Boole'un önerdiği ve 1938'de Claude Shannon'ın anahtarlama devrelerine uyguladığı Bool cebri üzerine kuruludur. Bu bölümde ikili mantığın aksiyomlarını, temel teoremlerini ve bunların fiziksel karşılığı olan mantık kapılarını inceleyeceğiz. Buradaki her kavram, önce Logisim'de simüle edilecek, sonra Verilog ile üç modelleme biçiminde yazılacak ve son olarak Icarus/Yosys akışıyla sentezlenecektir.

2.2 İkili Değişkenler ve Mantık Fonksiyonları

İkili Değişken: Yalnızca iki değer alabilen bir değişkene *ikili değişken* denir. Bu değerler genellikle $\{0, 1\}$, $\{\text{DOĞRU}, \text{YANLIŞ}\}$ ya da $\{\text{YÜKSEK}, \text{DÜŞÜK}\}$ gerilim seviyeleri olarak yorumlanır. Sayısal devre tasarımında 0 ve 1 gösterimi kullanılır.

Mantık Fonksiyonu: n adet ikili değişken $x_1, x_2, \dots, x_n$ alan ve tek bir ikili çıktı üreten $f : \{0, 1\}^n \rightarrow \{0, 1\}$ eşlemesine *mantık fonksiyonu* (Boole fonksiyonu) denir. Bir mantık fonksiyonu üç eşdeğer biçimde ifade edilebilir: **doğruluk tablosu**, **Bool ifadesi** ve **lojik devre şeması**. İlerleyen bölümlerde aynı fonksiyonu bu üç biçimde de üretip birbiriyle karşılaştıracğız.

Bilgi Notu 2.1: Neden Üç Eşdeğer Gösterim?

Doğruluk tablosu bir fonksiyonun *davranışını* kesin biçimde tanımlar ama büyüdükçe (2^n satır) okunması zorlaşır. Bool ifadesi kısa ve cebirsel işlemlere (sadeleştirme, De Morgan) uygundur. Devre şeması ise fiziksel gerçeklemeye en yakın gösterimdir. HDL tasarımında yazdığımız Verilog kodu da sonunda bu üçüncü forma, yani bir kapı ağına (netlist) sentezlenir.

2.3 Bool Cebriinin Aksiyomları

Bool cebri, Öklid geometrisi gibi *aksiyomatik* bir sistemdir: az sayıda temel varsayımdan (aksiyom) yola çıkılır ve cebriin tüm teoremleri bu varsayımlardan mantıksal çıkarımla türetilir. George Boole'un 1854'teki özgün formülasyonu, matematikçi E. V. Huntington tarafından 1904'te altı *postülat* indirgendi. Bu postülatlar, ikisi toplama (+) ikisi çarpma ($\cdot$) için olmak üzere dört çiftten oluşur; bu eşleşme, ilerleyen kısımda göreceğimiz *düalite ilkesinin* kaynağıdır.

Huntington Postülatları: $B = \{0, 1\}$ kümesi, + (OR) ve $\cdot$ (AND) ikili işlemleri ile ' (NOT) tekli işlemi altında, her $x, y, z \in B$ için aşağıdaki altı özdeşliği sağlıyorsa *Bool cebri* adını alır:

Tablo 2.1: Huntington Postülatları

No	Postülat	Postülat (düali)	Adı
1	$x + 0 = x$	$x \cdot 1 = x$	Birim eleman
2	$x + x' = 1$	$x \cdot x' = 0$	Tümleme
3	$x + y = y + x$	$x \cdot y = y \cdot x$	Değişme
4	$x + (yz) = (x + y)(x + z)$	$x(y + z) = xy + xz$	Dağılma

Bilgi Notu 2.2: Anahtarlama Devresi Yorumu

Shannon'ın 1938'de gösterdiği gibi, bu postülatlar fiziksel bir anahtarlama devresi olarak okunabilir: $x + y$ iki anahtarın **paralel** bağlanması (biri kapalıysa akım geçer), $x \cdot y$ **seri** bağlanması (ikisi de kapalı olmalı), x' ise x 'in tersidir (biri kapalıyken diğeri açıktır).

Örnek 2.1: Postülat 2'nin Anahtarlama Devresiyle Doğrulanması

x ve x' tanım gereği her zaman zıt durumdadır: biri kapalıyken diğeri mutlaka açıktır.

- **Seri bağlantı** ($x \cdot x'$): Akım geçmesi için *ikisinin de* kapalı olması gerekir. Zıt oldukları için bu hiçbir zaman gerçekleşmez $\Rightarrow x \cdot x' = 0$.
- **Paralel bağlantı** ($x + x'$): Akım geçmesi için *en az birinin* kapalı olması yeterlidir. Zıt oldukları için her zaman biri kapalıdır $\Rightarrow x + x' = 1$.

Alıştırma 2.1 ★ Postülat 1'i ($x + 0 = x$ ve $x \cdot 1 = x$) yukarıdaki örnekteki gibi anahtarlama devresi yorumuyla açıklayınız: 0 değerindeki bir anahtar her zaman açık, 1 değerindeki bir anahtar her zaman kapalıdır. Bu anahtarın devreye paralel ya da seri eklenmesi neden devrenin davranışını değiştirmez?

Çözüm: 0 değerindeki bir anahtar her zaman *açıktır*, yani hiçbir zaman akım geçirmez. Böyle bir anahtar x' 'e **paralel** eklendiğinde ($x + 0$), devreye ek bir akım yolu sağlamaz. Akımın geçip geçmemesi hâlâ tamamen x' 'e bağlıdır, dolayısıyla $x + 0 = x$. 1 değerindeki bir anahtar ise her zaman *kapalıdır*, yani her zaman akım geçirir. Böyle bir anahtar x' 'e **seri** eklendiğinde ($x \cdot 1$), akımı hiçbir zaman engellemez — yine akımın geçip geçmemesi tamamen x' 'e bağlı kalır, dolayısıyla $x \cdot 1 = x$.

Bilgi Notu 2.3: Sıradan Cebirden Fark

Postülat 4'ün ilk yarısı, $x + (yz) = (x + y)(x + z)$, sıradan cebirde **geçerli değildir** (ör. $x=2, y=3, z=4$ için sol taraf 14, sağ taraf 30 verir). Bool cebirinde geçerli olmasının nedeni, değişkenlerin yalnızca $\{0, 1\}$ değerlerini alması: bu sonlu kümede tüm $2^3 = 8$ kombinasyon tek tek denenerek özdeşlik doğrulanabilir. Bu ispat yöntemine *mükemmel tümevarım* (perfect induction) denir ve Bool cebirinde sıkça kullanılır.

2.4 Döualite İlkesi ve Temel Teoremler

Döualite İlkesi: Bir Bool özdeşliğinin *döuali*, ifadedeki her $+$ ile $\cdot$ 'nin, her 0 ile 1'in yer değiştirilmesiyle elde edilir. Postülatların her biri kendi döualiyle çift geldiği için (bkz. Tablo 2.1), bu çiftlenme **tüm türetilmiş teoremlere de yansır**: bir teoremi postülatlardan kanıtladığınızda, döualini kanıtlamak için hiçbir ek işlem gerekmez. Kanıttaki her satırda aynı yer değiştirmeyi uygulamak yeterlidir.

Bunu somutlaştırmak için, listedeki ilk teoremi doğrudan postülatlardan türetilim.

Örnek 2.2: Teorem 1'in Postülatlardan Türetilmesi: $x + x = x$

$$\begin{aligned}
x + x &= (x + x) \cdot 1 & [\text{Postülat 1: } y \cdot 1 = y, \text{ burada } y = x + x] \\
&= (x + x)(x + x') & [\text{Postülat 2: } 1 = x + x'] \\
&= x + x \cdot x' & [\text{Postülat 4: } (x + y)(x + z) = x + yz] \\
&= x + 0 & [\text{Postülat 2: } x \cdot x' = 0] \\
&= x & [\text{Postülat 1: } x + 0 = x]
\end{aligned}$$

Bilgi Notu 2.4: Döualitenin Getirisi: İkinci Teorem Bedava

Yukarıdaki kanıtın her satırında $+$ $\leftrightarrow \cdot$ ve $0 \leftrightarrow 1$ değişimini uygularsanız, $x \cdot x = x$ özdeşliğinin kanıtını elde edersiniz — tek bir ek adım atmadan. Bu yüzden Bool cebri kaynaklarında teoremler genellikle çift çift, tek bir kanıtla sunulur.

Aynı türetme yöntemiyle elde edilen diğer temel teoremler, ileride sadeleştirme işlemlerinde sürekli kullanacağımız için aşağıda topluca listelenmiştir.

Tablo 2.2: Temel Teoremler

No	Teorem	Teorem (düali)	Adı
1	$x + x = x$	$x \cdot x = x$	Yinelenirlik (idempotence)
2	$x + 1 = 1$	$x \cdot 0 = 0$	Yutan eleman
3	$(x')' = x$	—	Çifte tümlleme
4	$x + xy = x$	$x(x + y) = x$	Yutma (absorption)
5	$x + (y + z) = (x + y) + z$	$x(yz) = (xy)z$	Birleşme (associative)

Alıştırma 2.2 ★★ Teorem 4'ü ($x + xy = x$) yukarıdaki Teorem 1 kanıtını örnek olarak postülatlardan türetiniz. *İpucu:* x 'i önce $x \cdot 1$ olarak yazıp Postülat 4'ü (dağılma) tersten uygulamayı deneyin.

Çözüm:

$$\begin{aligned}
x + xy &= x \cdot 1 + x \cdot y & [\text{Postülat 1: } x &= x \cdot 1] \\
&= x \cdot (1 + y) & [\text{Postülat 4: } xy_1 + xy_2 &= x(y_1 + y_2), \text{ dağılma}] \\
&= x \cdot 1 & [\text{Teorem 2: } 1 + y &= 1] \\
&= x & [\text{Postülat 1: } x \cdot 1 &= x]
\end{aligned}$$

Terimler Sözlüğü

Taban (Radix) — Pozisyonel bir sayı sisteminde kullanılan basamak sayısı; her basamağın ağırlığı bu sayının kuvvetleriyle belirlenir (ör. ikili için 2, onaltılık için 16).

Ardışık Bölme/Çarpma Yöntemi — Ondalık bir sayıyı başka bir tabana çevirmek için kullanılan yöntem; tam sayı kısmı ardışık bölme (kalanlar aşağıdan yukarı okunur), kesirli kısım ardışık çarpma (taşan basamaklar yukarıdan aşağı okunur) ile çevrilir.

Tümleyen (Complement) — Bir sayının çıkarma işlemini toplamaya çevirmek için kullanılan dönüştürülmüş hâli; $(r-1)$ 'in tümleyeni (her basamağı $r-1$ 'den çıkararak) ve r 'nin tümleyeni (ona 1 eklenerek) olmak üzere iki türü vardır.

Uçtan-Dolaşan Elde (End-Around Carry) — $(r-1)$ 'in tümleyeniyle çıkarma yapıldığında oluşan taşma bitinin atılmayıp sonucun en düşük anlamlı basamağına eklenmesi kuralı.

İşaret Biti — İşaretli bir ikili sayıda en soldaki bit; 0 pozitif, 1 negatif anlamına gelir.

Taşma (Overflow) — n bitlik iki sayının toplamının n bite sığmaması durumu; donanımda sabit bit genişliği nedeniyle sonucun bozulmasına yol açar.

Normalizasyon — Bir kayan noktalı sayıyı $1.xxxx \times 2^k$ biçiminde, baştaki basamak tam olarak 1 olacak şekilde yazma işlemi; ikilide baştaki 1 her zaman var olduğu için saklanmaz (örtük bit).

Bias — IEEE 754'te üs alanına eklenen sabit kaydırma değeri (tek duyarlılıkta 127); negatif üsleri de işaretsiz tam sayı olarak karşılaştırılabilir kılar.

Bool Değişkeni — Yalnızca 0 veya 1 değerini alabilen değişken.

Mantık Kapısı — Bir veya birden çok ikili girişten tek bir ikili çıkış üreten temel devre elemanı.